

从零读懂 nsF5 隐写项目

信息隐藏 × 机器学习 × 工程实践

面向零基础本科生的项目学习手册：从像素与二进制出发，读懂 LSB 隐写、卡方与 RS 隐写分析、汉明矩阵编码、F5/nsF5 与湿纸编码、SHA-256 键控，再到监督式机器学习检测与 C++/GPU 工程加速，最后完成你自己的综合实验。

建议周期：6–12 个月（含 12 周快速入门） · 每周 6–8 小时

适合：刚接触 Python / 未系统学过信息安全与机器学习的同学

学习方式：先读原理 → 打开项目对照 → 亲手改代码做实验

2026 年 9 月 6 日 · 项目 v1.4.0 版 · 手册已随项目迭代至 v1.8.4

导读 · 这本手册怎么用

这是一份“跟着代码学”的手册。它不代替教科书，而是把 F:\Steganography 项目背后的信息隐藏与机器学习知识，按零基础本科生可以接受的顺序拆开：先用直觉和例子讲清楚“为什么”，再带你看看项目里“怎么实现”，最后让你亲手跑实验、改代码。

0.1 读者对象与起点

你只需要具备这些前提，其余内容手册会逐步补齐：

- 会用鼠标操作电脑、能安装软件（Windows 环境）；
- 上过大学数学公共课或愿意遇到公式时“先跳过、再回头”；
- 有一点编程基础更好，但没有也没关系——第 2 章会带你把 Python 用到够用；
- 至少有 12 周（约 3 个月）的耐心，每周 6–8 小时；想深入请预留 6–12 个月。

新手提示 |

如果你连 Python 都没装过，不要慌：前两周就是专门为这种情况设计的。你不需要成为程序员，只需要成为“能读懂并修改这个项目的人”。

0.2 本手册的阅读约定

手册里反复出现五种小方块，含义如下：

标记	含义
新手提示	把容易卡住的概念换成大白话或类比，让你顺利往下走
避坑提醒	初学者最容易踩的坑：数值类型、参数不一致、误读统计量等
动手做	必须亲自动手的小实验，只读不练等于没学
想一想	不需要标准答案的思考题，用来检验自己是否真的理解

标记	含义
回到项目看代码	指出应当打开哪个文件、读哪一段，建立“概念→代码”的映射

0.3 12 周快速入门路线总览（6–12 个月深入版见附录 F）

路线遵循“载体基础 → 隐写算法 → 隐写分析 → 机器学习 → 工程加速 → 综合 实践”的自然顺序。每一周都对应可运行的代码或可复现的实验：

周次	主题	对应章节	动手成果
第 1 周	数字图像与二进制	第 1 章	能亲手查看一张图的像素与位平面
第 1 周 (补充)	JPEG 与 DCT 系数	第 1½ 章	能解释量化表如何决定隐写容量
第 2 周	Python/NumPy/Pillow	第 2 章	跑通 GUI 或 run_e2e.py，能读写图像数组
第 3–4 周	LSB 隐写与盲分析	第 3 章	藏一句话→用卡方/RS 检测到它
第 5 周	矩阵编码与 F5	第 4 章	用汉明码做到“改动更少、藏得更多”
第 6 周	nsF5、湿纸与哈希键控	第 5–6 章	读懂 ns5_core.py，解释湿纸求解
第 7 周	机器学习基础	第 7 章	能讲清特征/标签/过拟合/AUC
第 8–9 周	机器学习隐写检测	第 8 章	跑通 v1 11 维与 v2 143 维两套管线；理解 SRM 与 143d/53d 双版本模型
第 10 周	C++/GPU/数据工程/GUI	第 9 章	看懂加速原理、自校验机制与多源数据集管线 (SRM/BOSSbase)
第 11– 12 周	综合项目与汇报	第 10–11 章	完成一个可演示的改进实验

动手做 |

周次	主题	对应章节	动手成果
请把这张表抄成一张打卡表贴在桌前，或直接使用附录 C 的检查表。每周结束时勾掉一行，并回答该章末尾的“想一想”。			

0.4 学完以后，你应该能做到

- 用自己的话解释：隐写、隐写分析、嵌入效率、收缩、湿纸编码、伴随式；
- 说出 LSB 隐写为什么会被卡方检验与 RS 分析发现；
- 手算一个 $p=3$ 的汉明矩阵嵌入例子（块内最多改 1 位）；
- 解释 nsF5 与 F5 的本质区别，以及为什么“无收缩”更重要；
- 解释图像哈希键控如何实现解码自同步，并说明它的局限；
- 解释机器学习二分类的完整流程，包括为什么要按照片分组交叉验证；
- 读懂 v1 的 11 维与 v2 的 143 维特征（含 SRM 统计），能解释 143d 稳健版与 53d 可解释版的双模型策略；
- 讲清 C++ DLL 与 GPU 加速分别加速了什么，为什么需要一致性校验；
- 独立完成一个小的改进实验并写出诚实的结果分析。

0.5 项目地图：先认识你要学的代码

整个项目是一个“研究工具”：既能做算法实验，也打包成带 GUI 的软件。先记住这些关键文件，后面每一章都会回来找它们：

文件	角色	学习顺序
src/image_io.py	图像读写封装：打开/保存 8bit 灰度与彩色图	第 2 章
src/ns5_core.py	算法核心：汉明码、湿纸、哈希键控、嵌入/解码	第 3–6 章
src/steganalysis.py	盲隐写分析：卡方、RS、综合概率与判读	第 3 章

文件	角色	学习顺序
src/efficiency.py	码族与嵌入效率理论/实测曲线	第 4 章
src/matrix_demo.py	伴随式查找的教学演示逻辑	第 4 章
src/fsfeatures.py + cpp/fsfeatures.dll	v1 11 维统计特征的 ctypes 绑定 (v2 143 维见 featurize_v2.py)	第 8 章
src/make_dataset.py / train_model.py	数据集生成 (v1/v2 特征、SRM、多源) 与训练/评估	第 8 章
src/ml_predict.py	单图 ML 判定封装 (143d/53d 双版本 + 灵敏度)	第 8 章
src/cppembed.py + cpp/nsf5embed.dll	C++ 嵌入/置乱热路径与自校验	第 9 章
gpu/*.py	PyTorch 批量特征 (v1 11 维 / v2 143 维) 与 GPU 训练	第 9 章
src/gui.py	tkinter 图形界面, 含教学面板	第 9 章
src/test_core.py 等测试	验证嵌入往返、分析与误报的回归测试	全程
src/featurize_v2.py + src/srm_filter.py + gpu/featurize_v2_gpu.py	v1.4 新增: 30 核 SRM 预处理与 143 维 v2 特征 (CPU/GPU 双实现)	第 8-9 章

回到项目看代码 |

建议从现在起保持项目窗口打开。手册里提到某个文件时, 就切过去找到它; 第 2 章之后, 多数知识点都要求你 “先运行、再阅读” 。

0.6 学习小贴士

- **先跑通, 再读懂。** 很多概念在纸上绕, 跑一次端到端脚本就通了;
- **公式分两步消化。** 第一遍只看结论和直觉, 第二遍再回到推导;
- **用自己的话复述。** 每章小结能讲给同学听, 才算掌握;
- **记录实验日志。** 改了哪些参数、结果如何, 是最后综合项目最宝贵的材料;

- **善用测试。** 项目自带的 `test_core.py` / `test_steg.py` 是最快的“对不对”裁判。

想一想 |

在你开始之前，先花 5 分钟回答：加密和隐写有什么不同？如果你说不清，太好了——这正是第 3 章要解决的第一个问题。

0.7 v1.4.0 更新说明 (2026-09-06)

本手册 9 月 3 日初稿对应项目 v1.3；9 月 6 日项目更新到 v1.4.0，本版手册已同步。隐写算法部分（第 1–6 章）不受影响；机器学习与工程部分补充了 8.8 与 9.7 两节，并把旧指标标注为“v1 基线”。

- ML 检测从 11 维特征 + LR/XGB (AUC 约 0.75–0.79) 升级为 143 维 LGB 默认版 (8-split 平均 0.8980) 与 53d 可解释版 (0.8461)：143d 更准、53d 每一维都能解释，详见 8.8 与 docs/RESULTS.md；
- 新增 SRM 高通滤波预处理、143 维 v2 特征与 12 档变体，并补入真实 JPEG 干净样本与 BOSSbase 全量多源数据，详见 8.8 与 9.7；
- 项目迁移到新 GitHub 仓库 (Yukinoshita-lin/nsf5-steganography)，许可证改为 Apache-2.0 并新增 NOTICE；
- 术语表、命令速查、12 周打卡表与概念→代码定位表已同步补充 v1.4 内容。

第 1 章 · 数字图像与二进制（第 1 周）

动手做 |

本章目标：把“图像 = 数字矩阵”的直觉焊死；会手算二进制与字节；能说出为什么人眼和统计都能容忍一点“像素级扰动”。

1.1 一张数字图像，其实就是一堆数字

把一张 **8bit 灰度图** 放大到像素级，你会看到它是一张由 0~255 整数组成的二维表格。0 代表全黑，255 代表全白，中间是不同深浅的灰。计算机只保存这张表格的“尺寸”和“数字”，不保存你肉眼看到的画面。

彩色图则是三张这样的表格叠在一起：R（红）、G（绿）、B（蓝）三个通道。常见的 RGB 图每个像素有 3 个 0~255 的值。比如一个橙色像素可以写成 R=255、G=128、B=0。

在代码里，灰度图就是一个二维数组，彩色图就是一个三维数组。项目的 `src/image_io.py` 把这件事封装成了两个函数：`load_as_gray()` 负责把任意支持的图片读成灰度数组，`save_image()` 负责把数组保存成 PNG。

新手提示 |

数学里我们用“行、列”描述矩阵，图像处理里则常说“高、宽”：一张 512×512 图就是 512 行、512 列。数组形状 (512, 512) 的写法正是“行数在前”。

1.2 二进制、字节与位

计算机底层只认识 0 和 1。一个 0 或 1 叫 **1 bit (位)**，8 个 bit 组成 **1 byte (字节)**。8 个 bit 一共能表示 $2^8=256$ 种组合，正好对应灰度值 0~255。

例如十进制 $200 = 128 + 64 + 8$ ，写成 8 位二进制是 `11001000`；它的**最低有效位 (LSB)**就是最右边的那个 0。把 200 的 LSB 改成 1，得到 201——在人眼里，灰度 200 和 201 几乎无法区分。

“最低有效位”是整本手册最重要的概念之一：LSB 隐写，就是偷偷修改像素的 LSB，把秘密比特藏进去。

动手做 |

打开一个 Python 交互环境（或写个小脚本），验证下面这段：

体验：十进制、二进制与 LSB

```
v = 200
print(bin(v))           # 0b11001000
print(v & 1)            # 最低位 = 0
print(v ^ 1)            # 翻转最低位 = 201
print(v & 0b11111110)   # 把最低位清零 = 200
print(v & 0b11111110 | 1) # 先清零再置 1 = 201
```

1.3 为什么图像里“有空地”可以藏东西

图像里藏着大量**冗余**，这要从两个层面理解：

- **视觉冗余**：人眼对微小亮度差、高频细节不敏感。把某个像素从 128 改成 129，几乎没人看得出来；
- **统计冗余**：自然图像相邻像素高度相关，大部分信息集中在低频。JPEG 等压缩格式就是靠去掉这些冗余才把文件变小；
- **LSB 平面近似随机**：自然照片的最低几位看起来“像噪声”，这恰好提供了天然的“掩护环境”。

于是出现了两类使用冗余的方式：**压缩**想把冗余去掉让文件变小，**隐写**想在冗余里塞秘密而尽量不破坏视觉与统计特征。二者争夺的是同一片“可修改空间”，这也是为什么很多隐写算法要针对特定压缩格式设计。

避坑提醒 |

不要以为“看不见 = 安全”。肉眼看不见只是最弱的要求；真正专业的检测者用统计工具，LSB 直接替换会留下非常明显的统计痕迹，这正是第 3 章卡方检验与 RS 分析能抓住它的原因。

1.4 位平面：把一张图拆成 8 层

把灰度值写成 8 位二进制后，可以按“第几位”把图像拆成 8 张二值图，称为 **位平面**。最高位（MSB）平面决定画面大轮廓；最低位（LSB）平面几乎全是细碎的噪声点。

这带来一个直觉：**LSB 平面原本就接近随机，往里面再塞一点伪随机秘密比特，肉眼和简单直方图都很难察觉。**但也正是这种“随机化”，改变了图像内在的结构，可以被更聪明的统计方法测出来。

数一数像素与 LSB 的取值（项目里到处是这样的 numpy 操作）

```
import numpy as np
img = np.asarray([[200, 201], [128, 129]], dtype=np.uint8)
print(img.shape, img.dtype)  # (2, 2) uint8
print(img & 1)               # LSB 位平面 [[0,1],[0,1]]
print(np.unpackbits(np.array([200], dtype=np.uint8)))
# [1 1 0 0 1 0 0 0] ← 最高位在前
```

1.5 回到项目：先认识 cover 与 stego

隐写领域里有两个固定称呼：没有秘密的原始载体叫 **cover（载体）**，藏入秘密后的图像叫 **stego（含密图）**。项目里这两个词随处可见：`img/cover.png` 是演示用载体，`output/stego_*.png` 是嵌入后的含密图。

回到项目看代码 |

打开 `src/run_e2e.py` 第 1 步：它用 numpy 生成一张 256×256 的平滑“封面图”并保存到 `img/cover.png`。仔细看生成方式——`np.linspace` 制造渐变背景，再加一点小噪声。理解这段代码，你就理解了一张自然感图像是如何从数字“长”出来的。

1.6 小结与自我检查

- 灰度图 = 0~255 整数矩阵；彩色图 = 三个通道矩阵；

- LSB 是像素值二进制的最低位，翻转它对视觉影响极小；
- 压缩消除冗余，隐写利用冗余，二者针锋相对；
- cover = 原始载体，stego = 含密图。

想一想 |

一张纯黑色（全 0）的图适不适合做 LSB 隐写的载体？一张纯随机噪声图呢？把想法写下来，学完第 3 章再回来对照。

第 1½ 章 · JPEG 是什么，DCT 系数是什么？（压缩域隐写的战场）

新手提示 |

这一章写于 v1.9.0 (2026-10)，是应"学习路径应该先讲清楚 JPEG 和 DCT，再进入 LSB/隐写"的反馈补进来的。它编号是"1½"而不是把后面各章整体重排：手册已有大量"见第 N 章"的交叉引用、10 个按章 Notebook 和事实校验脚本，重编号的收益远小于风险。你只需要在读完第 1 章之后、进入第 3 章之前，把这一章读完。

第 1 章把图像拆成了像素与位平面，接下来按理就该讲 LSB 隐写了。但本项目的主线算法 nsF5 有两个"家"：主仓库 `ns5_core.py` 里的像素域实现（改灰度像素），以及 v1.9.0 起接入主线的 JPEG 压缩域实现（改量化 DCT 系数，来自姊妹项目 `yccstego`）。教科书里的 F5/nsF5 讲的是后者。不先弄懂 JPEG 和 DCT 系数，第 4、5 章里的"+3→+2"就永远只是口号。

1.5.1 JPEG 是什么：一台"有损压缩机"

JPEG 不是一种文件格式那么简单，它是一条**有损压缩流水线**，对每个 8×8 像素块依次做三件事：

1. **DCT 变换**：把 8×8 的像素值改写成 64 个"花纹权重"（下一节细讲）；
2. **量化**：每个权重除以量化表里对应位置的一个步长，再四舍五入取整——**有损就发生在这一步**：右下角（高频）的步长大，除完取整后大量直接变成 0；
3. **熵编码**：把剩下的非零系数按之字顺序串起来做压缩编码，写成 .jpg 文件。

第 3 步是可逆的（能一字不差地解回来），第 2 步不可逆（取整丢掉的信息回不来）。**JPEG 压缩域隐写的全部艺术，就是让秘密住进第 2 步的产物——量化之后的系数——里。**

避坑提醒 |

这也解释了第 3 章 FAQ 的那句话"JPEG 会破坏像素 LSB"：一张藏了像素 LSB 的 PNG 被另

存为 JPEG 时，整条流水线重新跑一遍，量化取整会把 LSB 结构搅乱。反过来，JPEG 域隐写的是量化系数本身，编码器会带着这些系数原样做完熵编码——所以它的含密图**必须是 JPEG 自己**，而且**不能再被任何工具重新另存**（重存等于重新量化，载荷即毁）。GUI 对 JPEG 域含密图按原始字节保存，就是这个原因。

1.5.2 DCT 系数是什么：8×8 块的"配方表"

把 8×8 块看成一幅微型画像。DCT（离散余弦变换）准备了 64 种固定的"花纹"：左上角的花纹是整块的均匀明暗（最低频），往右下走花纹越来越细密（高频）。DCT 变换做的事，就是回答一个问题：**这一块像素，按每种花纹各用多少比例调配出来？**得到的 64 个配比就是 DCT 系数。

- 左上角第一个系数叫 **DC**（直流），表示整块的平均亮度，不参与隐写；
- 其余 63 个叫 **AC**（交流）系数；
- 自然图像的能量集中在左上（低频），所以量化后右下角大批系数归 0——**归零的系数不是载体，活下来的非零 AC 系数才是。**

动手做 |

下面的片段用项目压缩域实现里同一套 DCT 矩阵与量化表（yccstego/dct.py），对同一个 8×8 块分别按质量 60、85、95 量化，数一数非零 AC 载体和 $|c|=1$ 的"湿点"各有多少。跑之前请确认已安装压缩域依赖：`pip install yccstego`。

```
import numpy as np
from yccstego import dct as jdct

# 造一个 8x8 块：平滑渐变(低频为主) + 轻噪声(高频细节)
rng = np.random.default_rng(3)
block = np.linspace(90, 200, 8)[None, :] + np.linspace(0, 60, 8)[: , None]
block = np.clip(block + rng.integers(-8, 9, (8, 8)), 0, 255).astype(float)

counts = {}
for q in (60, 85, 95):
    T = jdct.scale_qtable(jdct.LUM_QT, q)      # libjpeg 风格的质量缩放
```

```

F = jdct.dct_blocks(block - 128)          # 中心化后做 DCT (JPEG 的做法)
Q = np.round(F / T).astype(int)           # 量化: 有损就发生在这一步
ac = Q.reshape(-1)[1:]                     # 去掉 DC, 剩 63 个 AC
nz = int(np.sum(ac != 0))
wet = int(np.sum(np.abs(ac) == 1))
counts[q] = nz
print(f"质量 {q}: 非零 AC 载体 {nz} 个, |c|=1 湿点 {wet} 个")
assert counts[95] > counts[60], "质量越高, 活下来的载体应越多"
print(f"质量 95 的非零 AC 载体多于质量 60: {counts[95]} > {counts[60]}")

```

观察 |

你会看到两个规律：质量从 60 提到 95，非零 AC 载体明显变多（量化步长变小，更多系数“活了下来”）；同时 $|c|=1$ 的系数不少——它们马上就要给你惹麻烦了。

1.5.3 在系数上做 nsF5：载体、湿点与减幅

JPEG 域 nsF5 的载体系数是**量化 Y 块（亮度）的非零 AC 系数**，每个系数贡献一个比特： $x = c \& 1$ （奇偶）。往一个比特里写 1，不是把系数翻转，而是 **F5 的“减幅”：幅值减 1、保符号**。

原系数	修改后	说明
+3	+2	幅值减 1，奇偶从 1 变 0
-5	-4	保符号，同样翻 LSB
+1	湿点，不碰	再减就归 0（收缩），系数会从载体集合里消失
-1	湿点，不碰	同上

为什么“收缩”是灾难：提取端要按同样的顺序收集载体系数，一个系数从 +1 缩成 0 就从集合里消失，后面所有比特错位。这正是 F5 的老毛病，也是 nsF5 用**湿纸编码**（把 $|c|=1$ 的系数划为“湿点”、只在干点上解方程）修掉的问题——第 5 章展开。

动手做 |

三行代码看懂"减幅"与"湿点"的边界：

```
for c in (3, -5, 1, -1):
    action = c - (1 if c > 0 else -1) if abs(c) > 1 else "湿点，不碰"
    print(c, "->", action)
```

想一想 |

1. +3 和 +2 的二进制 LSB 都是 1，减幅之后 LSB 怎么会变？——提示：想想 +1、-1 与奇偶的关系，再想 $|c|=1$ 时为什么不能减。
2. 与"翻转 LSB"（+3 与 +4 之间跳）相比，减幅让系数的幅值直方图在 $|c|=1$ 处堆积。这种堆积是伪装的破绽还是伪装的代价？第 8 章的"magnitude-1 指纹"会反过来用它抓隐写。

避坑提醒 |

JPEG 域的口令键控与像素域同理：隐藏路径由"色度系数哈希 + 口令"派生，解码端 p / 口令 / 质量（若重嵌）任一不一致都提取不出来。重跑契约两域一致：yccstego 0.2.0 起湿纸求解种子由输入派生，相同输入逐字节复现；档案记录建档时的 yccstego 版本，重跑同版本走字节级校验，跨版本自动退回"提取一致"（旧档兼容），这写在了 `src/experiment.py` 的文档里。

1.5.4 在真 JPEG 上跑一遍

前面两段是在"单块"上看原理。真正的嵌入由 yccstego 完成：整图分块、逐块量化、在全部载体上做伴随式编码、再熵编码回标准 .jpg。下面用桥接层把它跑通（GUI 的"JPEG 域"和 CLI 的 `-jpeg` 走的的就是这同一份代码）：

```
import sys
sys.path.insert(0, "src")          # 源码运行时需要；pip 安装后可删掉这行
import numpy as np
import jpegstego
```

```
rng = np.random.default_rng(7)
y = np.linspace(0, 220, 128)[None, :]
x = np.linspace(0, 90, 128)[:, None]
gray = np.clip(110 + y + x + rng.integers(-6, 7, (128, 128)), 0, 255).astype(np.uint8)
rgb = np.stack([gray, gray, gray], axis=-1)      # JPEG 域吃 RGB

jpg, rep = jpegstego.embed_jpeg(rgb, "你好, DCT 域!", p=3, quality=85)
print("改动系数:", rep["carriers_changed"], "容量(bit):", rep["capacity_bits"])

msg, _, tampered, head_match = jpegstego.extract_jpeg(jpg, p=3)
assert msg == "你好, DCT 域!" and not tampered and head_match
print("往返一致:", msg)
```

命令行等价形式 (安装后可用) :

```
nsf5stego embed cover.png --jpeg -m "秘密" --quality 85
nsf5stego extract cover_stego.jpg --jpeg
```

1.5.5 回到项目看代码

概念	文件	优先阅读内容
桥接层 (统一入口/ 降级提示)	src/jpegstego.py	embed_jpeg / extract_jpeg
块级 nsF5 (减幅 + 湿纸)	yccstego/yccstego/nsf5.py	_embed_block / solve_wet_paper
DCT 与量化表	yccstego/yccstego/dct.py	_dct_matrix / scale_qtable / LUM_QT
JPEG 域分析 ($ c =1$ 指纹)	yccstego/yccstego/steganalysis.py	analyze_y
重跑契约 (两域差 异)	src/experiment.py	verify() 与模块文档
GUI 域切换	src/gui.py	_on_domain_change /

概念	文件	优先阅读内容
		_do_embed_jpeg

想一想 |

把同一张 128×128 的图分别用像素域（第 5 章的 nsF5Pixel）和本章的 JPEG 域嵌入，统计"可写位置"的数量：像素域约有 16000 个像素位，JPEG 域只有几千个系数——是什么决定了两者的差距？质量 60 时再算一次呢？这个容量直觉会在第 10 章的综合实验里派上用场。

第 2 章 · 准备工具：Python、NumPy 与图像读写（第 2 周）

动手做 |

本章目标：装好环境、跑通端到端脚本，会用 NumPy 读写图像数组。从此以后，你做的每个实验都能立刻在项目里验证。

2.1 安装环境

项目依赖很轻：核心只需要 NumPy 与 Pillow，机器学习部分再用 scikit-learn、joblib 等。建议用较新的 Python 3.9+（README 示例使用 3.14，低版本同样可用）。

在项目根目录安装并做第一次自检

```
cd F:\Steganography
pip install numpy pillow
python src\test_core.py      # 核心算法自测
python src\test_steg.py     # 隐写分析自测
python src\run_e2e.py        # 端到端：嵌入→解码→分析→绘图
```

避坑提醒 |

如果系统默认 python 没有 tkinter，GUI 无法启动。README 给出的办法是使用带 tkinter 的解释器（例如 C:\Python314\python.exe）；只跑命令行脚本则不受影响。遇到 ModuleNotFoundError 时先确认你安装到了“运行它的那个解释器”里：python -m pip install ... 更稳妥。

2.2 用 NumPy 思考图像

NumPy 数组的核心概念只有四个：**形状 (shape)**、**类型 (dtype)**、**切片 (slice)** 与**向量化运算**。隐写算法本质上都在做：取出数组 → 按某种顺序改写部分元素的 LSB → 存回数组。

四个核心概念的 10 行速览

```
import numpy as np
a = np.arange(12).reshape(3, 4)    # 3 行 4 列
print(a.shape, a.dtype)            # (3,4) int64
```

```

b = a[:, 1]                # 取第 2 列
c = a[0:2, 0:3]            # 左上角 2x3 块
x = np.zeros((64, 64), dtype=np.uint8)
x[10:20, 5:15] = 255      # 画一个白色方块
print((x & 1).sum())       # 统计 LSB=1 的数量

```

注意 `dtype=np.uint8`：图像像素是 0~255 的 8 位无符号整数。做加减时要小心**溢出回绕**：

`np.uint8(200) + 100` 会得到 44 而不是 300。项目在翻转 LSB 时用“异或 1”而不是“ ± 1 ”，正是为了避免把 0 减成负数或把 255 加溢出。

2.3 图像读写：认识 `image_io.py`

project/src/image_io.py 的核心接口

```

from PIL import Image
import numpy as np

def load_as_gray(path):
    im = Image.open(path).convert("L") # 强制转灰度
    return np.asarray(im, dtype=np.uint8)

def save_image(image, path):
    im = Image.fromarray(np.ascontiguousarray(image))
    im.save(path)

```

回到项目看代码 |

自己打开 `src/image_io.py` 全文，注意 `load_image()` 与 `load_as_gray()` 的差异，以及 `SUPPORTED` 支持的扩展名。然后运行下面这段，读入项目自带的封面图：

动手：读图、看形状、存灰度版

```

import sys; sys.path.insert(0, "src")
import image_io as IO
img = IO.load_as_gray("img/cover.png")
print(img.shape, img.dtype)          # (256,256) uint8
print(img.min(), img.max(), img.mean())
IO.save_image(img, "output/my_copy.png")

```

2.4 第一次端到端运行：run_e2e.py

运行 `python src/run_e2e.py`，脚本会依次完成五件事：生成封面图 → 以 nsF5 p=3 嵌入一段英文消息（分别用空口令与口令）→ 解码比对 → 对干净图与含密图做盲隐写分析 → 绘制码族/效率图。

动手做 |

仔细阅读终端输出，找到三组数字并抄下来：嵌入比特数、改动像素数、干净图与含密图的分析概率。下一章你就能解释它们为什么不同。

你还看不懂嵌入内部不要紧——第 2 周的目标只是“让代码在你机器上跑起来”，以及建立“数组改 LSB”的直觉。

2.5 常见报错速查

报错 / 现象	原因	处理
ModuleNotFoundError	包装到了别的解释器	用运行脚本的解释器执行 <code>python -m pip install</code>
图像太小, 无法容纳头部+正文	像素数少于消息容量	换更大的图, 或调小 p / 缩短消息
UnicodeDecodeError / 乱码	默认只支持 ASCII 消息	嵌入文本保持 ASCII; 中文方案见第 10 章拓展思路
GUI 秒退 / TclError	解释器无 tkinter	换带 tkinter 的 Python 运行 <code>src/gui.py</code>

想一想 |

为什么 `image_io` 保存时要把数组转成“连续数组”（`np.ascontiguousarray`）？提示：切片与转置可能产生非连续内存视图，很多底层库不喜欢它们。

第 3 章 · LSB 隐写与它的统计“指纹”（第 3–4 周）

动手做 |

本章目标：亲手完成一次“藏进去→检测出来”的最小闭环；理解卡方检验与 RS 分析各自抓住了什么；看懂 steganalysis.py 的判读逻辑。

3.1 加密 vs 隐写：先回答第一个问题

维度	加密 (Encryption)	隐写 (Steganography)
保护对象	消息内容：没有密钥就读不懂	通信行为：别人看不出有秘密
可见性	密文明显 “不像正常数据”	载体看起来完全正常
典型场景	传输密码、证书、加密文件	隐蔽信道、版权水印、取证研究
相互关系	可先加密再隐写（强烈推荐）	隐写隐藏的是“秘密存在”这一事实

新手提示 |

一句口诀：**加密保护“内容”，隐写保护“存在”**。专业隐写系统通常两者都用：先把消息加密，再把密文藏进载体。加密后密文近似随机，反而更不容易在统计上露馅。

3.2 最小可运行的 LSB 隐写

最朴素的隐写叫 **LSB 替换 (LSB replacement)**：把秘密比特串逐位写进像素的 LSB。灰度图有 $M \times N$ 个像素，就能藏 $M \times N$ 个比特。

教科书版最小 LSB（仅用于理解；项目实际用的是第 4–5 章的矩阵编码）

```
import numpy as np

def text_to_bits(s):
    raw = s.encode("ascii")
    # 16 bit 长度头 + 每个字符 8 bit
    head = [(len(raw) >> i) & 1 for i in range(16)]
```

```

body = np.unpackbits(np.frombuffer(raw, np.uint8))
return np.concatenate([head, body]).astype(np.uint8)

def lsb_embed(cover, bits):
    flat = cover.ravel().copy()
    k = min(len(bits), flat.size)
    flat[:k] = (flat[:k] & 0xFE) | bits[:k] # 清最低位再写入
    return flat.reshape(cover.shape)

def lsb_extract(stego, nbytes):
    flat = stego.ravel() & 1
    return bytes(np.packbits(flat[16:16 + nbytes*8]))

```

避坑提醒 |

有损格式是 LSB 的天敌。PNG/BMP 无损保存，位面不会变；JPEG 有损压缩会把 LSB 打得面目全非。所以项目默认保存 PNG，而 JPEG 域隐写需要另做一套（姊妹项目 `ycdstego` 走的正是量化 DCT 系数路线：<https://github.com/Yukinoshita-lin/ycdstego>，独立仓库与 PyPI 包，不在本仓库内）。

为什么消息开头要先放 16 bit 长度头？因为解码端必须知道“读多少字节”才能还原消息。项目 `src/ns5_core.py` 的 `encode_string()` 正是先写 16 位小端长度、再写 UTF-8 字节——你在 3.2 的例子中已经把这个结构复制了一遍。

3.3 为什么会露馅：卡方检验

自然图像里，相邻灰度 $2i$ 与 $2i+1$ （例如 100 与 101）出现的次数通常**不相等**。LSB 替换会强制把偶数与奇数“配对拉平”：当某像素从 $2i$ 改成 $2i+1$ 或反过来，两个灰度的频数会趋向均衡。Westfeld 的卡方检验做的就是这件事：

1. 统计整张图的 256 级灰度直方图，按相邻对 (0,1)、(2,3)、...、(254,255) 配对；
2. 假设“已嵌入”，则每对的两个频数应近似相等，期望值取对之和的一半；
3. 计算统计量 $\sum(\text{观测} - \text{期望})^2 / \text{期望}$ （只统计频数和大于 0 的灰度对）；
4. 把统计量换算成 p 值：p 高表示“观测与均匀假设吻合” → 疑似 LSB 随机化；

5. 项目还会把图切成 20 个前缀段分别计算 p 值，取中位数，得到更稳的判据。

注意 p 值的含义容易反：这里 p 越大越可疑，因为它在说“如果 LSB 已被完全随机化，看到这种分布的几率很大”——而干净的平滑图通常不会这么均匀。

避坑提醒 |

天然噪声图会骗过卡方检验。 数码照片本身 LSB 就接近随机，卡方 p 天然很高。所以项目引入了“内容本底随机度”修正：先用灰度差分熵估计图像本身有多‘随机’，再决定是否把高 p 当作嵌入证据。这是把启发式做成可用的关键工程细节。

3.4 RS 分析：LSB 的“结构塌缩”

Fridrich 等人的 RS 分析换了一个角度：不比较灰度对频数，而是观察 LSB 位面的**空间结构**。做法是把像素流切成 4 像素一组，定义一个平滑度判别式 f（组内相邻差绝对值之和），再分别用“正掩码”与“负掩码”翻转组内部分像素的 LSB，统计翻转后 f 变大（规则 R）还是变小（奇异 S）的组数。

干净的自然图像在负掩码下有明显的“常规倾向”，常用量 $G_n = (R_n - S_n)/N$ 显著为正（项目里干净平滑图常在 0.3 ~ 0.7）。LSB 替换让位面随机化后，这个结构“塌缩”， G_n 明显下降。项目同时输出 G_r 、估计嵌入率等，共同组成“RS 证据”。

项目 *steganalysis.py* 中 RS 的核心逻辑（伪代码，仅示意分组与统计思想）

```
groups = flat[:m].reshape(-1, 4)          # 每 4 像素一组
fg = np.abs(np.diff(groups, axis=1)).sum(axis=1) # 原平滑度 f
pos = (groups ^ np.array([0,1,1,0])) & 0xFF    # +M 翻转
neg = (groups ^ np.array([1,0,0,1])) & 0xFF    # -M 翻转
Rm = (f(pos) > fg).sum(); Sm = (f(pos) < fg).sum()
Rn = (f(neg) > fg).sum(); Sn = (f(neg) < fg).sum()
Gn = (Rn - Sn) / n                          # 干净图明显为正，随机化后≈0
```

3.5 项目的综合判读：analyze()

src/steganalysis.py 的 `analyze(image, sensitivity=...)` 并不只看一个 统计量，而是组合四路信号：

- 卡方中位 p 值（判断灰度对是否被拉平）；
- RS 塌缩幅度（Gn 相对内容自适应基线的下降比例）；
- 灰度差分熵与 LSB 差分熵（判断“随机”是不是图像本底自带）；
- 前缀 p 值序列（观察随机化是局部还是整体）。

三档灵敏度（严格/均衡/宽松）调整“可能/高度可能”的判定阈值与概率牵引系数，本质是在**误报与漏报**之间选择操作点。

3.6 动手实验：藏一句英文，再抓它出来

利用项目真实 API：嵌入 → 保存 → 分析（可在 Jupyter 中逐段执行）

```
import sys; sys.path.insert(0, "src")
import numpy as np
import image_io as IO
from ns5_core import embed_string, extract_string
import steganalysis as SA

clean = IO.load_as_gray("img/cover.png")
stego, report, nbits = embed_string(
    clean, "Hello nsF5!", method="nsF5", p=3, password="")
IO.save_image(stego, "output/lab3_stego.png")
print("改动像素:", report["cover_changed"])
print("解码:", extract_string(stego, method="nsF5", p=3))
for name, im in (("干净", clean), ("含密", stego)):
    r = SA.analyze(im)
    print(name, "Gn=%.3f" % r["RS_Gn"],
          "chi2p=%.3f" % r["chi2_pvalue"],
          "prob=%.2f" % r["stego_probability"], r["verdict"])
```

动手做 |

把消息改成 5000 个字符再跑一次（参考 `src/test_steg.py`），观察改动像素占比、Gn 下降幅度与隐写概率的变化。再换一张噪声较多的照片，看看卡方与 RS 是否还能区分。

3.7 小结与自我检查

- LSB 替换会拉平相邻灰度对频数（卡方可测）并破坏位面结构（RS 可测）；
- p 值高 $\neq$ 安全；要先判断图像本底是否天然随机；
- 误报/漏报不可兼得，灵敏度设置就是选择操作点；
- 项目 `analyze()` 是 “统计信号 + 内容基线” 的工程化组合。

想一想 |

纯 LSB 替换 “藏 1 bit/像素”，改动率平均 50%。如果只藏少量消息、改动不到 1% 像素，卡方还查得到吗？RS 呢？带着这个问题进入第 4 章。

第 4 章 · 矩阵编码与 F5：少改一点，藏得更多（第 5 周）

动手做 |

本章目标：理解“嵌入效率”为什么重要；能手算 $p=3$ 的汉明伴随式嵌入例子；知道 F5 的“收缩”问题出在哪。

4.1 从“每像素 1 位”到“平均每次改动藏 p 位”

朴素 LSB 每藏 1 bit 就平均改动 0.5 个像素，嵌入效率很低。如果有一种编码：每读 n 个像素组成的块，就能通过**最多改动 1 个像素**写入 p 位消息，改动次数就会大幅下降。这个想法叫 **矩阵嵌入 (Matrix Embedding)**，它用到的数学工具是二元汉明码。

定义嵌入效率 α = 平均每次修改所携带的比特数。理论极限是每次修改最多携带 p 位， $\alpha(p) = p \cdot 2^p / (2^p - 1)$ ： $p=2$ 时约 2.67， $p=3$ 时约 3.43， $p=4$ 时约 4.27，随 p 增大而上升，但需要的块长 $n=2^p-1$ 也在变大。

新手提示 |

为什么 p 越大“越划算”？因为汉明码用 $n=2^p-1$ 个位置承载 p 位消息； p 越大， n 相对 p 的“浪费”越少，平均改 1 次能寄的信息越多。代价是块变大、可嵌入消息对载体结构更挑剔。

4.2 汉明码速成：校验矩阵与伴随式

设块长 $n=2^p-1$ 。构造一个 $p \times n$ 的**校验矩阵 H** ，它的 n 列恰好是 $GF(2)^p$ 里的全部非零向量： $p=3$ 时，七列就是二进制 1..7。把块内 n 个像素的 LSB 写成列向量 x ，定义伴随式 $s = H \cdot x \pmod{2}$ ，它是一个 p 位向量。

嵌入 p 位消息 m 时，我们想通过翻转个别位，让新伴随式等于 m 。因为 H 的每一列都不同，翻转第 j 位只会把伴随式变成 $s \oplus H_j$ ——也就是说，**任意需要的伴随式变化都对应某一行**。

1. 计算当前伴随式 $s = H \cdot x$;

2. 若 $s == m$, 块内无需改动;
3. 否则求差值 $d = s \oplus m$ (p 位非零向量);
4. 在 H 中找到等于 d 的那一列, 记为第 j 列;
5. 翻转第 j 个像素的 LSB, 完成 ($H \cdot x' = m$)。

4.3 手算一个 $p=3$ 的例子

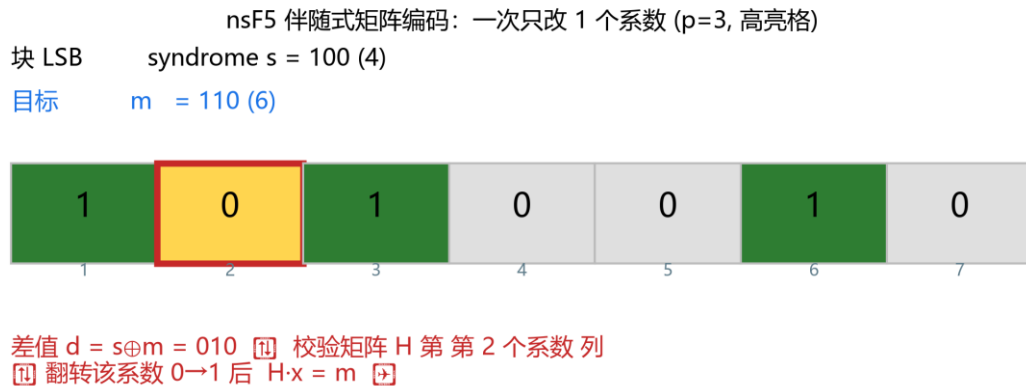
设块内 7 个像素的 LSB 为 $x = [0,1,0,1,1,0,1]$ 。列向量按二进制顺序 (低位在上): $H_1=[1,0,0]$, $H_2=[0,1,0]$, $H_3=[1,1,0]$, $H_4=[0,0,1]$ 依此类推。先算伴随式: x 中取 1 的位置是第 2、4、5、7 列, 做异或: $H_2 \oplus H_4 \oplus H_5 \oplus H_7 = [0,1,0] \oplus [0,0,1] \oplus [1,0,1] \oplus [1,1,1] = [0,0,1]$, 所以 $s=[0,0,1]$ 。

想嵌入 $m=[1,1,0]$: $d = s \oplus m = [0,0,1] \oplus [1,1,0] = [1,1,1]$ 。 $H_7=[1,1,1]$, 命中第 7 列, 于是只翻转第 7 个像素的 LSB: $x'=[0,1,0,1,1,0,0]$ 。再算一次 $s'=H \cdot x'=[1,1,0]=m$, 成功。

7 个像素承载 3 位消息, 但平均只需要改动 $(2^3-1)/2^3 \approx 87.5\%$ 的块里改 1 位——约 0.875 次/块, 折算每 3 位约 0.875 次改动, 即每次改动约带 3.43 位。

动手做 |

项目里 `src/matrix_demo.py` 把这个过程变成了可点击的可视化面板 (GUI 的 “矩阵编码演示” 标签页)。先用 `demo_step(p, x, m_bin)` 看返回的字典里 s 、 d 、命中的列号, 再手工验证 $H \cdot x' == m$ 。

图 4-1 伴随式矩阵编码示意：块内至多修改 1 个系数（项目图， $p=3$ ）

4.4 F5：矩阵编码 + JPEG 系数减幅

F5 是 Westfeld 提出的 JPEG 隐写：在量化 DCT 交流系数上做矩阵嵌入。修改“位”不是翻转 LSB，而是让系数的**绝对值减小 1**（例如 $+3 \rightarrow +2$ 、 $-5 \rightarrow -4$ ），这样既改变 LSB 奇偶，又不容易引入显著统计异常。

问题出在“收缩”：当绝对值已经是 1 的系数被减幅，会直接变成 0。而 0 系数在 JPEG 解码里意味着“没有这个频带分量”，F5 认为它不再是合法载体，于是丢掉这块、重试后续消息。**收缩**导致实际载荷变小、修改效率低于理论值，还留下可被统计的特征。

4.5 回到项目：MatrixEmbedding 与效率曲线

回到项目看代码 |

打开 `src/ns5_core.py` 第 216 行附近的 `MatrixEmbedding` 类：`_embed()` 正是 4.3 步骤的直接代码——计算 s 、比较 m 、找列、翻转。`src/efficiency.py` 则画出了理论 $\alpha(p)$ 与实测效率的对比图。

MatrixEmbedding._embed() 核心（真实代码节选）

```
x1 = (c[pos] & 1).astype(np.uint8)    # 取块内 LSB
s = syndrome(H, x1)                    # s = H·x
if np.array_equal(s, m): continue       # 恰好命中，不改
```

```
d = (s ^ m).astype(np.uint8)          # 差值
col = int(np.where(np.all(H == d[:, None], axis=0))[0][0])
c[pos[col]] ^= 1                       # 翻转命中像素
```

生成并查看码族/效率图

```
import sys; sys.path.insert(0, "src")
from efficiency import plot_code_family_and_efficiency
path = plot_code_family_and_efficiency(p_max=6)
print(path)  # output/efficiency.png
```

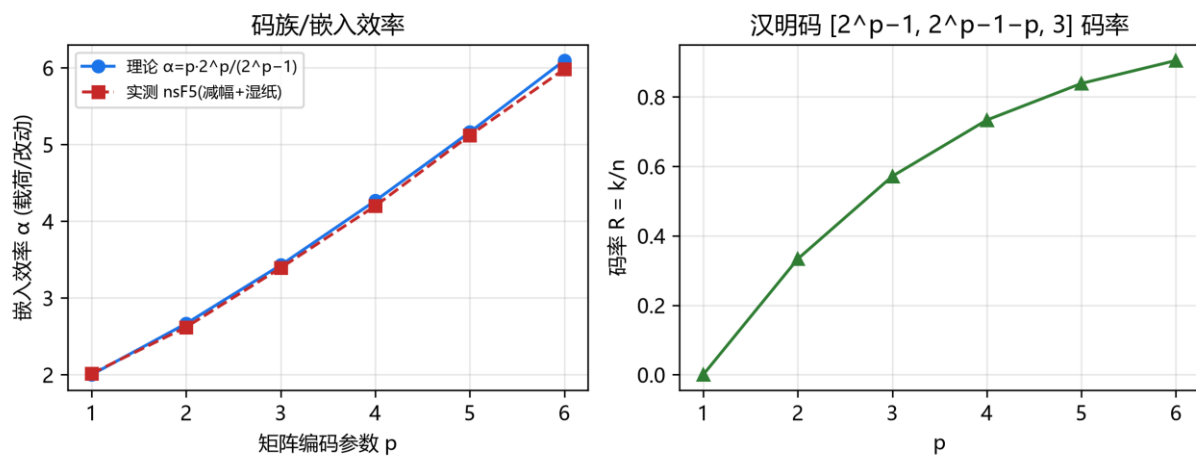


图 4-2 汉明码码族与嵌入效率：理论 vs 实测（项目图）

4.6 小结与自我检查

- 矩阵嵌入以“块”为单位： n 个位置承载 p 位、至多改 1 位；
- H 列互不相同 $\Rightarrow$ 任意伴随式差值都能定位到唯一需要翻转的位置；
- 嵌入效率 $\alpha = p \cdot 2^p / (2^p - 1)$ ，随 p 增加但块长也增加；
- F5 在 JPEG 系数上减幅嵌入，遇到 $|\text{系数}|=1$ 减到 0 就“收缩”。

想一想 |

如果 F5 遇到收缩就“跳过重试”，消息还可能正确解码吗？解码端如何知道哪块被跳过？想不通没关系——第 5 章的 nsF5 用湿纸编码彻底绕开了这个难题。

第 5 章 · nsF5 与湿纸编码：项目的核心算法（第 6 周）

动手做 |

本章目标：说清“收缩”为什么伤效率；理解湿纸编码“湿点不动、干点解方程”的思想；能对照 ns5_core.py 讲出 nsF5 嵌入的完整流程。

5.1 先把问题拧清楚：F5 的收缩

F5 在 JPEG 量化系数上做减幅修改：系数绝对值每被减 1，LSB 奇偶就翻转一次。但绝对值 = 1 的系数（例如 +1、-1）再减就变成 0。对 JPEG 来说，0 系数是“缺席的频带”，不适合继续当载体，F5 只能放弃这一块重新寻找位置——这就是**收缩 (shrinkage)**。

- 收缩让**实际容量**低于理论值：块被浪费，消息需要更多位置；
- 收缩让**嵌入效率**低于 $\alpha(p)$ ：为补偿损失需要更多次修改；
- 收缩还会留下可检测的统计特征：被改到 0 的系数变多，直方图出现异常。

项目在空间域（像素）上做了一个巧妙类比：把像素值减去 128 变成“有符号值” $xv = \text{像素} - 128$ ，于是 $|xv| \leq 1$ 的像素（127、128、129）就是“减幅会撞到零附近”的危险位置。矩阵嵌入在 JPEG 上的收缩难题，在像素域里变成了“127/128/129 不能作为普通修改对象”的同一问题。

5.2 湿纸编码：先划湿点，再解方程

湿纸编码 (Wet Paper Coding) 是 Fridrich 等人提出的框架，用来回答一个问题：如果载体里有一部分位置“碰不得”（像被水浸湿的纸，写了也看不见），能不能只用其余“干”位置完成消息嵌入？

答案是可以，而且不需要通信双方预先商量哪些位置是湿的——只要算法能从图像内容本身重建出同样的湿/干划分即可。项目正是这样做的：

1. 对每个块，先找出“减幅会变成零/危险”的位置，标记为湿点；

2. 真正的问题是：在**干位置**集合上找一个修改向量 e ，使翻转这些位置后整块的伴随式恰好等于目标 m ；
3. 即解方程 $H[:, \text{干}] \cdot y = d$ ，其中 $d = s \oplus m$ 是期望的伴随式变化；
4. 项目按“单个干列命中 $\rightarrow$ 两个干列异或命中 $\rightarrow$ GF(2) 高斯消元兜底”的顺序求解，尽量少改；
5. 解出的 y 是 0/1 向量： $y=1$ 的位置就是实际要减幅的干位置。

新手提示 |

为什么“先标记湿点”就不收缩了？因为收缩的根源是“改到一半才发现改成了 0”。nsF5 把所有会撞零的位置提前排除，只在保证安全的干点上求解，于是每个块都能成功嵌入，既不用重试，也不会出现多余的 0 系数。

5.3 读懂项目：nsF5Pixel._embed()

src/ns5_core.py 里 nsF5Pixel 类继承自 MatrixEmbedding，只重写了 _embed()。逐行读这段代码，你会看到完整的 nsF5 决策树：

nsF5Pixel._embed() 决策树（伪代码流程，逻辑与原代码一致）

```
xv = c[pos].astype(np.int16) - 128      # 像素→有符号系数
s = syndrome(H, (xv & 1).astype(np.uint8))
if s == m: continue                      # 伴随式已匹配
tc = 命中列(H, s ^ m)                   # 最“直接”的候选位置
if abs(xv[tc]) > 1:                       # 减幅不会撞零 → 直接改
    c[pos[tc]] -= sign(xv[tc])           # 向 128 靠拢一步
else:                                    # 候选位置是湿点
    dry = [k for k in range(n) if abs(xv[k]) > 1]
    e = solve_wet_paper(H, dry, d)       # 只在干点上解方程
    if e: 按 e 减幅所有命中干点
    else: 兜底翻转目标位 LSB（保证可解码）
```

5.4 再看三个求解函数

函数	作用	关键点
----	----	-----

函数	作用	关键点
<code>solve_wet_paper(H, dry_cols, target)</code>	在干列里找尽量稀疏的解	权重 1 → 权重 2 → 高斯兜底
<code>gauss_solve_GF2(cols, b)</code>	GF(2) 高斯消元解线性方程组	自由变量置 0 压低改动数
<code>permute_index(total, seed)</code>	确定性伪随机置换	splitmix64 + Fisher-Yates, 可 C++ 加速

回到项目看代码 |

打开 `src/ns5_core.py` 第 145–215 行，对照 5.4 的表格逐段读。建议先读函数注释，再用 `src/test_core.py` 的 `test_wet_paper_needed()` 验证：它把像素强制设成 127/128/129 制造湿点，仍能完成嵌入/解码往返。

5.5 验证实验：效率与往返

运行核心自测 + 对比两种方法

```
python src\test_core.py
# 对比 matrix 与 nsF5 改动像素数：
import sys; sys.path.insert(0, "src")
import numpy as np; from ns5_core import embed_string
img = np.random.default_rng(0).integers(0, 256, (64, 64), np.uint8)
for method in ("matrix", "nsF5"):
    stego, rep, nb = embed_string(img, "Hello nsF5!", method=method, p=3)
    print(method, "改动", rep["cover_changed"], "像素 / 容量", nb)
```

对同一张随机图、同一条消息，`matrix` 与 `nsF5` 的改动数不一定谁更少——因为 `nsF5` 的保护范围（127/128/129）在随机图上很少触发。真正的差异出现在“危险像素 密集”的图上：`nsF5` 依然能无收缩完成嵌入，`matrix` 则可能反复失效。

5.6 小结与自我检查

- 收缩 = 减幅撞零导致块作废，损伤容量、效率与统计安全；
- 湿纸编码把“不能改的位置”预先划为湿点，只在干点解伴随式方程；

- nsF5 = 矩阵嵌入 + 湿纸编码，无收缩、无重试；
- 项目在像素域用 $xv = \text{像素} - 128$ 模拟 JPEG 系数， $|xv| \leq 1$ 视为湿点。

想一想 |

为什么湿纸编码“不需要预先约定湿点位置”？解码端怎么知道哪些像素不能用来承载消息？提示：解码端只需要读 LSB 算伴随式，它不需要知道嵌入时绕过了谁。

第 6 章 · 口令、SHA-256 与“键控”安全设计（第 6–7 周）

动手做 |

本章目标：理解为什么嵌入位置要“由密钥决定”；会讲清解码自同步与篡改感知的原理；知道这套设计的边界在哪里。

6.1 如果嵌入位置是固定的，会怎样？

假设每次都是从左上角开始顺序嵌。攻击者知道算法后，可以：① 直接读取前 N 个 像素的 LSB 还原消息；② 把消息原位复制到另一张“看起来相同”的图像上（位置 替换攻击）；③ 对固定区域做针对性扰动，既破坏你的消息又不改动其余图像。

因此现代隐写会把嵌入路径做成**密钥控制的伪随机顺序**：没有口令（或不知道 内容哈希）的人，不知道消息在哪一块，也无法进行位置级攻击。

6.2 两条种子规则：先认图，再找正文

项目把像素分成两个池：**头部池**（开头 N_h 个块）与**正文池**（其余像素）。密钥派生用两个独立的种子：

- **seed0 = 仅由口令派生**：决定头部池的置乱顺序。头部里预埋的是 cover_hash——由原图像字节计算的 SHA-256 截断前 16 字节（128 位）；
- **seedB = cover_hash + 口令联合派生**：决定正文池的置乱顺序与分块。

解码时，接收方先用自己的口令重建 seed0，读回头部取出 cover_hash，再与口令一起重建 seedB，才能找到正文。口令错误或头部被破坏，后续路径立刻失效——test_pwd_mismatch() 测的就是这一点。

项目中的种子派生（真实代码）

```
def derive_seed(image_bytes, password=""):
    base = hashlib.sha256(image_bytes).hexdigest()
    if password:
        base = hashlib.sha256(
```

```
base.encode() + password.encode()).hexdigest()
return int(base, 16)
```

6.3 确定性置乱：splitmix64 + Fisher-Yates

有了种子还不够，还要把它变成“1..N 的一个确定排列”。项目用 splitmix64 作为伪随机源，执行 Fisher-Yates 洗牌：从末尾向前，每一步用伪随机数选一个位置交换。同样的种子必然产生同样的排列——这正是“自同步”的数学基础。

回到项目看代码 |

打开 `src/ns5_core.py` 的 `permute_index()`：DLL 可用时调用 C++ 的 `nsf5_permute`，否则走 Python 同算法 fallback。**两套实现必须给出完全相同的排列**，否则用 DLL 嵌入的图在无 DLL 的机器上无法解码。`src/cppembed.py` 的 `selfcheck()` 专门做这种跨语言一致性校验。

6.4 篡改感知：到底能感知什么

嵌入端在报告中给出 `cover_hash`（原图哈希）。任何像素被改动，图像字节的 SHA-256 都会改变。把收到的图像重新计算哈希，与嵌入端报告的（或头部解出的）哈希比较：不一致 = 图像被动过。更妙的是，由于正文路径依赖这个哈希，即使攻击者只改了一个像素，正文置乱种子也随之错位，解码结果会立刻崩溃或乱码。

同时要诚实地说清边界：这套机制是键控（keying）与篡改感知，不是密码学意义上的完整性认证。头部哈希本身没有用密钥做 MAC，理论上有能力重嵌整张图的攻击者可以同步更新头部哈希。README 把这个列为后续工作（例如引入密钥化 MAC），阅读时注意区分“检测普通篡改”与“抵御恶意重嵌”。

6.5 动手实验：口令错误与像素篡改

实验 A：口令不匹配应失败；实验 B：改 1 像素后解码

```
import sys; sys.path.insert(0, "src")
import numpy as np; import image_io as IO
```

```
from ns5_core import embed_string, extract_string, get_image_hash

img = IO.load_as_gray("img/cover.png")
stego, rep, _ = embed_string(img, "secret", method="nsF5",
                             p=3, password="A")
print(extract_string(stego, method="nsF5", p=3, password="B"))
tampered = stego.copy(); tampered[0, 0] ^= 1  # 只改 1 个像素
print(get_image_hash(stego)[:16], "vs", get_image_hash(tampered)[:16])
print(extract_string(tampered, method="nsF5", p=3, password="A"))
```

避坑提醒 |

实验 B 的失败方式可能是 `ValueError`、乱码或截断消息，取决于被改像素落在哪块。不要期待“总是抛异常”——篡改感知的工程表现就是：与哈希比对失败，且正文无法自同步。

6.6 小结与自我检查

- 固定路径 = 可预测 = 可被复制与针对；键控路径让隐藏位置由口令+内容决定；
- 头部放 128 位内容哈希，正文种子依赖该哈希 → 解码无需外部同步；
- 确定性置换（splitmix64 + Fisher-Yates）是自同步的数学保证；
- 键控 ≠ 完整性认证；强对抗场景需要密钥化 MAC。

想一想 |

如果两张不同的原图被嵌入了同一条消息，含密图的隐藏位置会一样吗？为什么？（提示：正文种子由什么决定？）

第 7 章 · 机器学习基础（写给第一次学的人）（第 7–8 周）

动手做 |

本章目标：建立“数据→特征→模型→评估”的完整框架；能讲清过拟合、交叉验证与数据泄漏；看懂 AUC 与阈值选择。

7.1 隐写检测，本质上是一个二分类问题

把整件事换一个视角：给定一张图，它要么是干净载体（label=0），要么藏过消息（label=1）。机器学习的任务，就是学一个函数 $f(\text{图像}) \rightarrow 0/1$ 或“含密概率”。这正是项目里监督式隐写检测做的事。

整个监督学习只有四样东西：**数据、特征、模型、评估**。下面按这个顺序讲，每个概念都立刻对应到项目代码。

7.2 数据：样本、特征与标签

v1 把每张图压缩成 11 个数字，叫特征向量；v1.4 又在其上扩展出 143 维 v2 特征（见 8.8）。先以 11 维教学：所有样本组成一张表 data/dataset.csv，目前有 2898 行 = 414 张照片 ×（1 个干净 + 6 个含密变体），其中干净 414 个、含密 2484 个。

数据集长什么样（前 3 列是 11 维特征的一部分）

```
Rm,Sm,Rn,Sn,RS_Gr,RS_Gn,chi2_pvalue,diff_entropy,lsb_diff_entropy,median_prefix_p,chi2_stat,
label,photo_id,...
41316,4574,36608,5057,0.56,0.48,3.5e-278,1.99,0.986,2.7e-166,1719.3,0,0,...
40488,4907,35701,5342,0.54,0.46,2.0e-269,2.02,0.989,2.2e-155,1675.9,1,0,...
```

data/dataset.csv（真实前两行，数值截断显示）

注意 photo_id 这一列：同一张照片的干净版与各变体共享同一个 id。这个字段是防数据泄漏的关键（v1.4 的 v2 数据集扩到 12 档并混入真实 JPEG 干净样本，分组思想不变）。

7.3 模型怎么“学”：损失、梯度与逻辑回归

先看最简单的可解释模型——**逻辑回归**：它对特征做线性加权 $z = w \cdot x + b$ ，再用 sigmoid 函数 $\sigma(z) = 1/(1 + e^{-z})$ 压到 $0 \sim 1$ ，输出“含密概率”。

训练就是找一组权重 w ，使模型在所有样本上的**损失**最小。逻辑回归常用的损失是交叉熵：预测 1 而真值是 0，损失就大；预测越错，惩罚越陡。优化方法（如梯度下降）反复微调 w ，让损失一点一点降下来。

新手提示 |

你不必现在会推梯度公式。先记住三句话：**损失 = 模型错得有多离谱**；**训练 = 找一组参数让损失最小**；**梯度下降 = 沿着“损失下降最快的方向”一次次挪动参数**。项目用 scikit-learn 的 `LogisticRegression`，这些细节都被封装好了。

7.4 过拟合、交叉验证与数据泄漏

模型在“见过的数据”上表现好很容易，真正要测的是**没见过的数据**——这叫泛化。如果模型把训练样本背下来，在测试集上就会崩，这就是**过拟合**。

防止过拟合的标准流程是**交叉验证**：把数据切成若干折，轮流用其中一折当验证集。但这里有一个隐蔽陷阱：同一张照片的 7 个样本高度相似，如果随机切分，训练集里可能出现某照片的“兄弟样本”，模型等于提前看到了测试照片——这叫**数据泄漏**，AUC 会虚高得离谱。

避坑提醒 |

项目用 `GroupKFold`：**按照片 id 分组切分**，保证同一张照片的所有变体要么全在训练集、要么全在验证集。`src/train_model.py` 的 `cv_auc()` 只做这一件事，却决定了整个实验的可信度——这是本项目最值得学习的机器学习工程细节之一。

7.5 评估指标：别只看准确率

隐写检测的正负样本不平衡（干净少、含密多），且“误报一张干净图”和“漏掉一张含密图”代价不同，所以要用一组指标：

指标	回答的问题	直觉
准确率 accuracy	整体猜对比例	不平衡时容易骗人
精确率 precision	判为含密里有多少真含密	误报低→精确率高
召回率 recall	真含密里抓到多少	漏报低→召回率高
F1	精确率与召回率的调和平均	两者都想要时的折中
ROC / AUC	模型排序能力与阈值无关	AUC=0.5 等于瞎猜

模型输出的是概率，还要选一个**阈值**才能变成 0/1 判断。阈值越高越保守（误报少、漏报多）；阈值越低越激进（检出高、误报多）。项目训练时用 **Youden 准则**在“真阳率-假阳率”最大处选阈值，另外专门算一个“低误报点”阈值（假阳率 $\leq 10\%$ ），供严格档使用。

Youden 阈值（真实代码逻辑）

```
from sklearn.metrics import roc_curve
fpr, tpr, th = roc_curve(y_true, proba)
j = tpr - fpr                                # Youden J
best = float(th[np.argmax(j)])                # 离随机线最远的点
```

动手做 |

学完本节运行 `python src\train_model.py` 一次。你不需要读懂全部输出，先找到三行：CV-AUC、测试集 AUC、低误报点检出率。第 8 章将逐行解释它们的含义。

7.6 小结与自我检查

- 监督学习 = 用带标签样本学一个从特征到答案的函数；
- 随机切分在同源样本上会泄漏信息，GroupKFold 按组切分才是诚实的；

- AUC 衡量排序能力，阈值决定实际操作点；
- 误报率与检出率是一对矛盾，Youden/低误报点是两种取舍。

想一想 |

如果一份论文的隐写检测 $AUC=0.95$ 但没说怎么切分数据，你会先怀疑什么？把怀疑写下来，这就是你评估一切 ML 实验的起点。

第 8 章 · 用机器学习做隐写检测（第 8–9 周）

动手做 |

本章目标：理解“为什么是 11 维统计特征而不是裸像素 CNN”；认识每个特征在测什么；跑通数据生成→训练→评估→GUI 判定全链路。

8.1 先回答一个反直觉的问题：为什么不用 CNN 直接看像素？

深度学习直接吃像素（“裸 CNN”）在这个任务上并不好用。项目在 414 张照片上试过多架构深度卷积网络，验证 $AUC \approx 0.50$ ，即完全无法泛化。原因值得细品：

- 隐写改动的是**极弱的高频信号**（LSB 层），信噪比远低于物体、纹理等视觉内容；
- CNN 很容易学到“照片内容”而不是“隐藏痕迹”，一旦测试照片风格不同就失效；
- 真正独立的样本数量是照片数（几百张），而深度模型通常需要成千上万张独立源图。

于是项目选择“领域知识做特征 + 简单分类器”路线：手工设计能直接度量 LSB 统计足迹的特征，再用逻辑回归/树模型判别。v1 时代 11 维特征在小样本上的 AUC 约 0.75 ~ 0.79；v1.4 升级到 143 维 + LGB 双版本后达到 0.90+（见 8.8），但“特征法优于裸 CNN”的基本结论不变。

新手提示 |

这不是“深度学习无用”的结论，而是“在样本量约束下如何选建模策略”的方法论课：**先用可解释的强特征验证信号是否存在，再考虑更复杂的模型**。很多工程问题都遵循这个次序。

8.2 认识 11 个特征

v1 的 11 维特征全部由 C++ 库 `cpp/fsfeatures.dll` 计算（Python 绑定在 `src/fsfeatures.py`）；v2 的 143 维特征在 `src/featurize_v2.py` 中组装。按“测什么”可以分成三组：

特征	含义	嵌入后会怎样
----	----	--------

特征	含义	嵌入后会怎样
Rm / Sm / Gr	正掩码下常规/奇异组数与缺口	Gr 下降（结构受损）
Rn / Sn / Gn	负掩码下常规/奇异组数与缺口	Gn 显著塌缩（核心信号）
chi2_stat / chi2_pvalue	卡方统计量及其 p 值	灰度对趋向均匀，p 上升
diff_entropy	灰度差分熵：图像本底粗糙度	用于识别天然噪声图
lsb_diff_entropy	LSB 位面差分熵	随机化后趋近 1
median_prefix_p	20 个前缀段卡方 p 的中位数	整体随机化时升高

回到项目看代码 |

打开 `src/steganalysis.py`，找到 `diff_entropy()` 与 `lsb_diff_entropy()`：它们用相邻像素差值的香农熵估计“随机度基线”。再打开 `src/fsfeatures.py`，看 11 维特征是如何排列并交给 C++ 的。

8.3 数据怎么来：make_dataset.py 的“1+6”设计

`src/make_dataset.py` 对每张源照片生成 1 张干净图 + 6 个含密变体（v1.4 另支持 12 档 v2 变体，见 8.8）：

变体（方法, p, 密度）	强度	目的
clean	—	提供干净的负样本
nsF5, p=3, d=0.25	弱	贴近真实隐蔽通信的弱密度
nsF5, p=3, d=0.55	中	中等密度，检验一般强度
nsF5, p=2, d=0.35	中弱	p 越小效率越低，痕迹更强
nsF5, p=2, d=0.85	强	强嵌入，统计足迹明显
matrix, p=3, d=0.50	中	对比另一方法

变体 (方法, p, 密度)	强度	目的
matrix, p=2, d=0.80	强	普通矩阵编码的高密度
v1.4 再 +6 档	—	nsF5 p3 d0.40; matrix p3 d0.40/0.60; lsb d0.30/0.50/0.70

所有图统一缩放为 512×512 灰度；特征全部由 C++ 提取，含密变体由 C++ 嵌入路径生成，保证训练与推理一致。

8.4 训练脚本怎么读：train_model.py

python src\train_model.py 的流程是一条完整的“诚实评估流水线”：

1. 读表并删除特征缺失的行；
2. 按 photo_id 用 GroupShuffleSplit 预留 25% 作为最终测试集（本组照片绝不进训练池）；
3. 训练池上做 5 折 GroupKFold 交叉验证，比较逻辑回归 / 随机森林 / 梯度提升 / XGBoost；
4. 选出 CV-AUC 最高的模型，在训练池内再切 20% 作为校准集选阈值（Youden 与低误报点）；
5. 用全部训练池重训，在 held-out 测试集上报 acc / balanced acc / AUC / 分类报告；
6. 按 (method, p, density) 分组报告每档检出率；
7. 保存模型与阈值到 models/stego_classifier.joblib (v1.4 起支持 LGB 网格调优、Stacking 与 53d 可解释版另存)。

避坑提醒 |

校准集用于挑阈值，测试集只许用一次。如果反复拿测试集调阈值，测试集就“脏”了。项目用三层结构——训练/校准/测试，是工业界也认可的严谨做法。

8.5 怎么读结果：ROC、AUC 与按档检出率

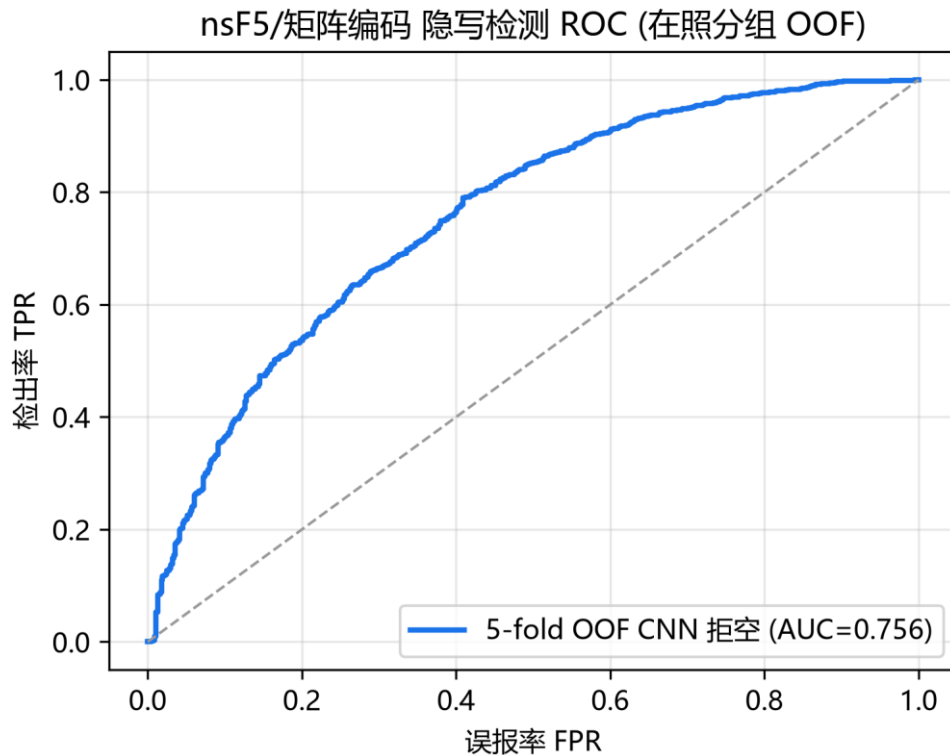


图 8-1 隐写检测 ROC 曲线 (v1 数据集·11 维基线：按照片分组留出外折, AUC $\approx$ 0.756)

ROC 曲线把阈值从 1 扫到 0，画出“检出率 vs 误报率”。AUC 是曲线下面积：0.5 是瞎猜。v1 基线上 0.75 量级说明“大部分时候能把含密图排在干净图前面”，但弱密度仍容易漏；v1.4 双版本模型（见 8.8）把 8-split 平均 AUC 做到 0.8980 (143d) / 0.8461 (53d)，口径见 docs/RESULTS.md。

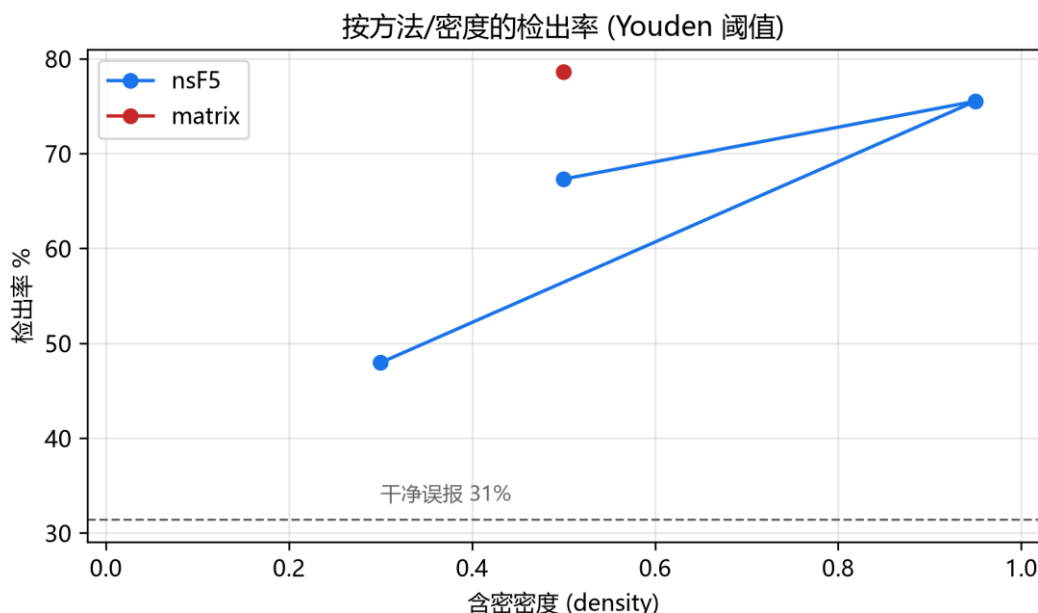


图 8-2 按方法/密度的检出率 (Youden 阈值; v1 数据集 6 档基线)：强密度高、弱密度明显下降

回到项目看代码 | 对比 experiments/data/density_grid.csv (第 5 章实验产出的真实统计)：lsb 各档几乎 100% 检出，matrix p2 d0.80 约 95%，而弱档 nsF5 p3 d0.25 只有约 47%、nsF5 p3 d0.55 约 55%。这说明“弱嵌入更难检测”不是玄学，而是统计足迹随密度衰减的直接结果。

8.6 灵敏度：在 GUI 里如何工作

src/ml_predict.py 的 MLPredictor 预处理好图像（转灰度→512×512）后，按模型记录的特征列数自动走 v1 11 维或 v2 143 维特征提取，再输出含密概率。灵敏度三档通过切换阈值工作：

灵敏度	ML 阈值	效果
严格（低误报）	threshold_low_fp	干净图误判最少，弱含密更容易漏
均衡	Youden 阈值	检出与误报平衡，默认档
宽松（高检出）	Youden×0.75	弱密度检出更多，误报略升

GUI 的“分析”页会把启发式概率与 ML 概率并排显示，要求你**交叉印证**，而不是只看一个数——这是项目刻意保留的诚实设计。

8.7 动手实验：从训练到单图判定

完整链路（v1 数据集；v2 与双版本命令见 8.8）

```
# 1) 生成数据集（可用自己的照片目录替代 F:\DCIM\Camera）
python src\make_dataset.py

# 2) 训练与评估
python src\train_model.py

# 3) 对单张图做 ML 判定
import sys; sys.path.insert(0, "src")
import numpy as np
from PIL import Image
from ns5_core import embed_string
from ml_predict import get_predictor
a = np.asarray(Image.open("img/cover.png").convert("L"))
s, _, _ = embed_string(a, "ML test", method="nsF5", p=3)
print(get_predictor().predict(s))
```

动手做 |

做一个“诚实性小实验”：先对一张照片做 ML 判定（应接近干净），再嵌入弱档消息（nsF5 p=3、密度 0.25 附近）重新判定。观察概率是否只微弱上升。这正是 README 里“弱密度需交叉印证”警告的现场体验。

8.8 v1.4.0 更新：SRM、143 维特征与双版本模型（2026-09）

8.1–8.7 描述的是 v1.0–v1.3 的路线：11 维特征 + LR/XGB，AUC 约 0.75–0.79。v1.4.0 沿三条线做了升级：SRM 高通滤波预处理、143 维 v2 特征与 12 档变体、以及最终的 LightGBM 双版本模型。下面按项目实际演进顺序讲，帮助你读懂 README 中那一长串实验表。

8.8.1 SRM 高通滤波：同源有收益，跨源反而亏

SRM（Spatial Rich Model）是 Fridrich 体系的一组高通滤波核。`src/srm_filter.py` 实现 30 个标准核（一阶差分 8 + 二阶 4 + 三阶 8 + 边缘 $3 \times 3/5 \times 5$ 共 8 + 方形 $3 \times 3/5 \times 5$ 共 2），对图

像逐核滤波后取最大绝对响应、clip 到 ± 4 再缩放成单通道“增强图”，喂给原有特征器。CPU (numpy) 与 GPU (torch) 双实现，接入开关是 `make_dataset.py --preprocess srm` 与 GPU 的 `use_srm=True`。

指标	同源原图基线	同源 SRM 增强
5 折 CV-AUC	0.7594	0.7922
held-out 测试 AUC	0.7811	0.8085
弱档 nsF5 p3 d0.25 检出	51.9%	62.5%
nsF5 p2 d0.35 检出	76.0%	90.4%
低误报点含密检出	41.2%	50.3%
干净误报 (低误报阈值)	9.6%	9.6%

避坑提醒 (v1 时期对照，数字未在当前版本复现) | 当年观察到：SRM 预处理的收益只在“同源”数据内出现。跨源合并 (校园 + BOSSbase 全量) 时，SRM 反而把 CPU 测试 AUC 从 0.741 拉到 0.704、GPU 验证 AUC 从 0.651 拉到 0.572——高通滤波在压掉图像内容的同时，也压平了不同相机/压缩源之间的可分性。因此默认模型采用未 SRM 的合并全量；SRM 单源校园模型另存为 `campus_srm` 专用。

8.8.2 v2 143 维特征与 12 档变体

`src/featurize_v2.py` 在 11 维基础上增加：30 个 SRM 残差的均值/绝对均值/标准差 (90 维)、20 段整图前缀卡方 p、20 段 LSB 前缀卡方 p，以及 `texture_noise` 与 `est_rate` (2 维)，合计 143 维。训练变体从 6 档扩展到 12 档：追加 nsF5 p3 d0.40、matrix p3 d0.40/0.60，以及 lsb d0.30/0.50/0.70。

严格 A/B (同一测试集照片分组、5 折 GroupKFold OOF)：v2 数据集本身带来 +0.019 (同 11 维时 0.7868 vs 0.7678)；在 v2 数据集上再加 143 维又 +0.027 (OOF 0.8143、held-out 0.8278)。累计约 +0.05 AUC。

8.8.3 真实 JPEG 干净图暴露的部署问题与修复

v2 LR 在训练分布内 AUC 很高，但部署到真实 JPEG 干净照片时几乎全判 1.0：SRM 残差对 JPEG 高频噪声过于敏感。修复办法是把 414 张真实 JPEG 干净图加进训练，得到 dataset_campus_v2_jpeg.csv，再做 LGB 网格调优。【2026-09-14 审计更正】这批 JPEG 干净行当初被赋予了独立 photo_id，而其特征与对应 clean 行逐位相同（是副本）：同一张源图的样本因此可以跨训练/验证两侧，“按源图划分”名存实亡、指标被抬高；同一批 GPU 特征还把像素先除以 255 再过高通滤波，与推理端 CPU 特征相差约 30 倍。两处都已修复并重跑（修复方式见 experiments/add_jpeg_clean.py，细节见 docs/RESULTS.md 与 CHANGELOG 1.6.2）。现口径：143d held-out AUC 0.8939、8-split 平均 0.8980、弱档 nsF5 p3 d0.25 检出 50.0%；53d 为 0.8391 / 0.8461 / 41.4%。

8.8.4 双版本部署策略：143d 稳健 vs 53d 可解释

可解释性实验（2026-09-14 按修正后的特征口径重算）：143 维里 SRM 90 维贡献了 52.6% 的 LightGBM gain，BASE 11 占 20.7%、LSB PREFIX 20 占 20.6%、PREFIX 20 占 6.2%；消融显示去掉 SRM 90 维后 5 折 OOF AUC 从 0.9010 掉到 0.8513（约 +0.05 AUC）。也就是说修好尺度之后 SRM 不是“噪声特征”。于是项目保留两套模型：

维度	143d 默认 (稳健)	53d 可解释
特征构成	BASE11 + SRM90 + PREFIX20 + LSB-PREFIX20 + TEX/EST2	去 SRM 后的 53 维
8-split 平均 AUC	0.8980	0.8461
Held-out AUC	0.8939	0.8391
弱档 nsF5 p3 d0.25	50.0%	41.4%
真实 JPEG 干净 OOD	9.58% (n=1514, 95%CI 8.20–11.16%)	28.86% (n=1514, 95%CI 26.64–31.20%)
推荐场景	通用部署 / 异构数据 / 真实图	教学 / 答辩 / 单图可解释 (AUC 略低约 0.05)

`MLPredictor()` 默认加载 143d；传入 `model_path` 可切 53d。推理时会按模型记录的特征列数自动选择 11 维或 143 维特征提取。GUI 的模型单选框仍在规划中，当前通过 API 切换：

v1.4.0 双版本模型切换示例

```
from ml_predict import MLPredictor

# 默认：143d 稳健版 (models/stego_classifier.joblib)
pred_143 = MLPredictor()

# 53d 可解释版 (去 SRM 90 维, AUC 略低、但可逐维解释)
pred_53 = MLPredictor(
    model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib',
    clip_outliers=False)

r = pred_53.predict(img)
# {'probability': ..., 'verdict': ..., 'threshold': ...}
```

动手做 | 实验任务：用 `$env:DS_FILES = "dataset_campus_v2_jpeg.csv"` 重新训练，分别用 143d 与 53d 对“真实 JPEG 干净图”和“弱档 nsF5 p3 d0.25 嵌入图”做判定，把 AUC 与 OOD 行为写进实验报告。

避坑提醒 | v2/53d 模型只在训练分布附近可靠：训练含校园照片（PGM/BMP 灰度派生）与已加入的 JPEG 干净样本。对全新相机/压缩源，先做小规模 OOD 测试再谈部署。

8.9 小结与自我检查

- 小样本 + 弱信号场景下，领域特征 + 简单模型通常比裸 CNN 更可靠；
- 11 个特征 = RS 家族 + 卡方家族 + 熵/随机度基线；
- 训练/校准/测试三层结构 + GroupKFold 是实验可信度的骨架；
- ML 概率与启发式判读应交叉印证，弱密度检测有其物理上限。
- v1.4 之后双版本模型在训练分布内已很强，但真实部署前仍必须做 OOD 验证：本项目在 1514 张真实干净照片（校园 414 + DIV2K 100 + ALASKA#2 1000）上实测，143d 误报 9.58% (95%CI 8.20–11.16%)、53d 28.86% (26.64–31.20%)；DIV2K 这类 2K 高清

图是主要失分来源（143d 约 51%）。评估脚本 `experiments/ood_eval.py`，结果见 `docs/RESULTS.md` 第 5 节。

想一想 |

数据集中同一张照片有 7 个样本，为什么不能用“随机 80/20 切分”？请设计一个最小例子说明泄漏会让 AUC 虚高多少。

第 9 章 · 工程化：C++、GPU、GUI 与测试（第 10 周）

动手做 |

本章目标：看懂“算法”如何变成“软件”；理解为什么需要 C++ 与 GPU 加速、如何保证跨语言一致；会运行测试与 CI 流程。

9.1 系统架构：分层而不耦合

系统总体架构（模块化分层）

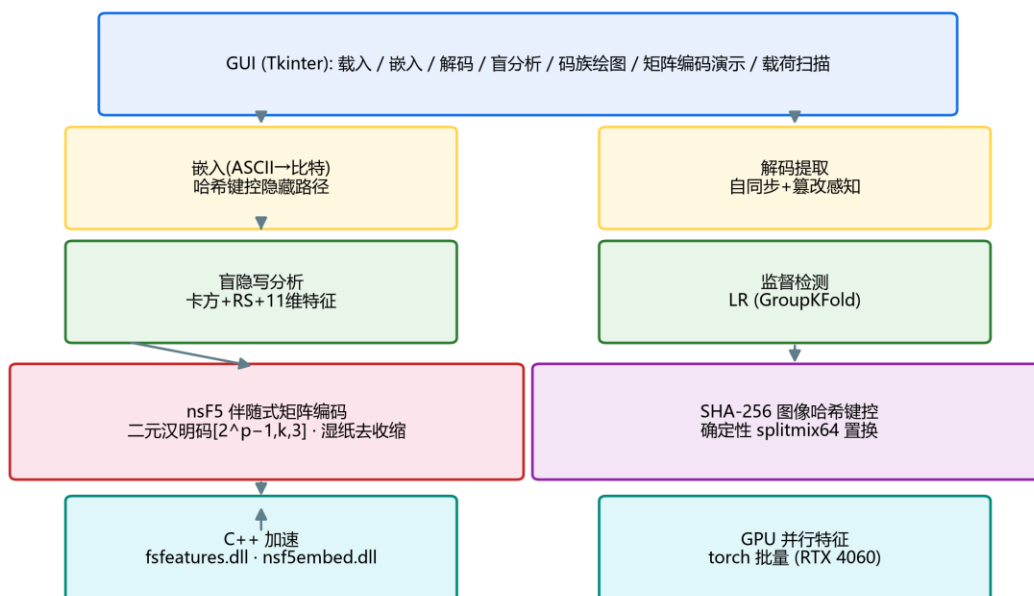


图 9-1 系统总体架构：底层加速 → 算法安全核心 → 功能层 → GUI（项目图）

- **加速层**：cpp/fsfeatures.dll（特征）、cpp/nsf5embed.dll（嵌入/置乱）；
- **算法核心**：ns5_core.py——汉明码、湿纸、哈希键控、嵌入/解码；
- **功能层**：steganalysis.py、efficiency.py、make_dataset.py、train_model.py、ml_predict.py；
- **界面层**：gui.py + matrix_demo.py + scan_panel.py，所有功能一键可点。

分层的意义：算法层不依赖 GUI，可以在无界面环境（服务器/CI）里跑；加速层 缺失时自动回退 Python，保证功能永远可用。

9.2 C++ 加速：为什么是“热路径”，以及如何保证没错

Python 逐元素循环极慢，但隐写有两个典型热路径：确定性置换（把 1600 万 像素的顺序洗一遍）与特征提取（对每张图做 RS/卡方/熵统计）。把它们下沉到 C++ 动态库，性能可以提升几个数量级：置换加速随规模变化（数据见 experiments/data/bench_permute.csv，可用 ``python experiments/tools/gen_bench.py --only permute`` 复现；性能与机器相关）：
 4096^2 (1600 万像素) Py ≈ 15.6 秒 $\rightarrow$ C++ ≈ 0.24 秒（约 65 倍），同一份基准里小规模（6.5 万）最高约 240 倍。。

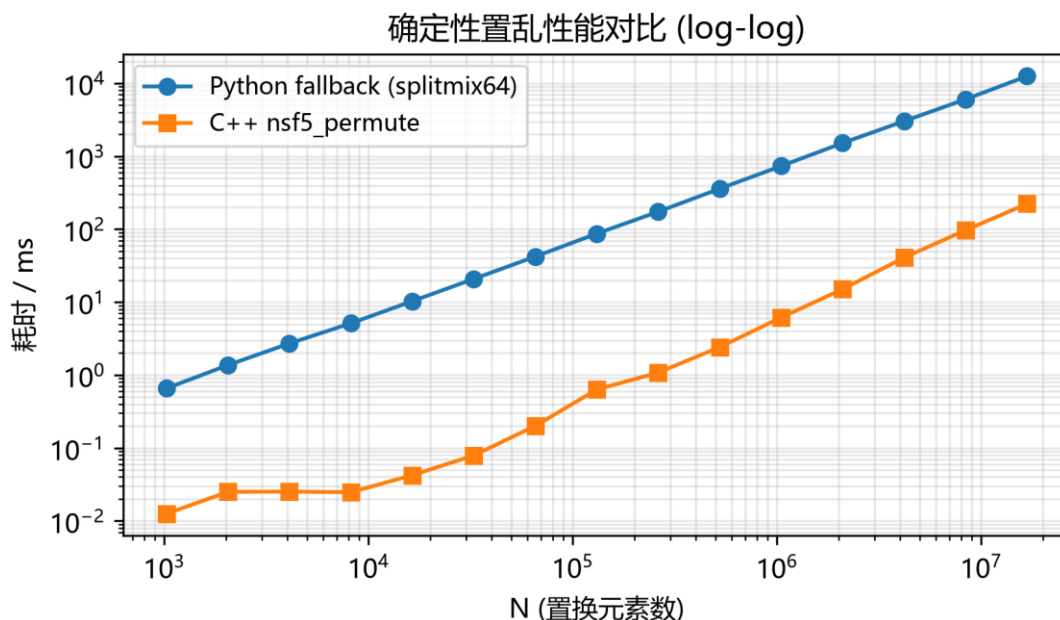


图 9-2 确定性置换性能：Python vs C++ (log-log 坐标，项目图)

性能提升的前提是“结果完全一样”。Python 调 DLL 用的是 ctypes，项目为此设计了自校验：

- `cppembed.py` 用同一图像分别走 C++ 与 Python 路径嵌入，比较输出是否像素级一致；
- `fsfeatures.py` 的 `check_against_python()` 逐特征核对 C++ 与 Python 结果；
- DLL 缺失或加载失败时自动回退 Python 同算法，保证嵌入/解码两端序列恒定可逆。

避坑提醒 |

跨语言一致性是“可加速”的前提，也是调试的雷区：C++ 端位运算、取整、溢出行为与 Python/NumPy 不完全一致。遇到“有 DLL 能嵌、无 DLL 解不了”的诡异 bug，先跑 `cppembed.selfcheck()` 和 `fsfeatures.check_against_python()`，而不是去改算法。

9.3 GPU 版：把 11 维特征“批量向量化”

gpu/ 目录用 PyTorch 把特征计算改写成张量算子：一次读入一批 512×512 灰度图，在 GPU 上并行完成直方图、RS、卡方与熵计算，再落回 CPU。v1.4 新增 `gpu/featurize_v2_gpu.py` 支持 143 维 v2 特征（含 SRM 卷积）。README 实测 v1 特征 2070 张约 5 秒（ ≈ 410 张/秒），且与 CPU 参考实现逐位一致（浮点误差 $\sim 1e-7$ ）。

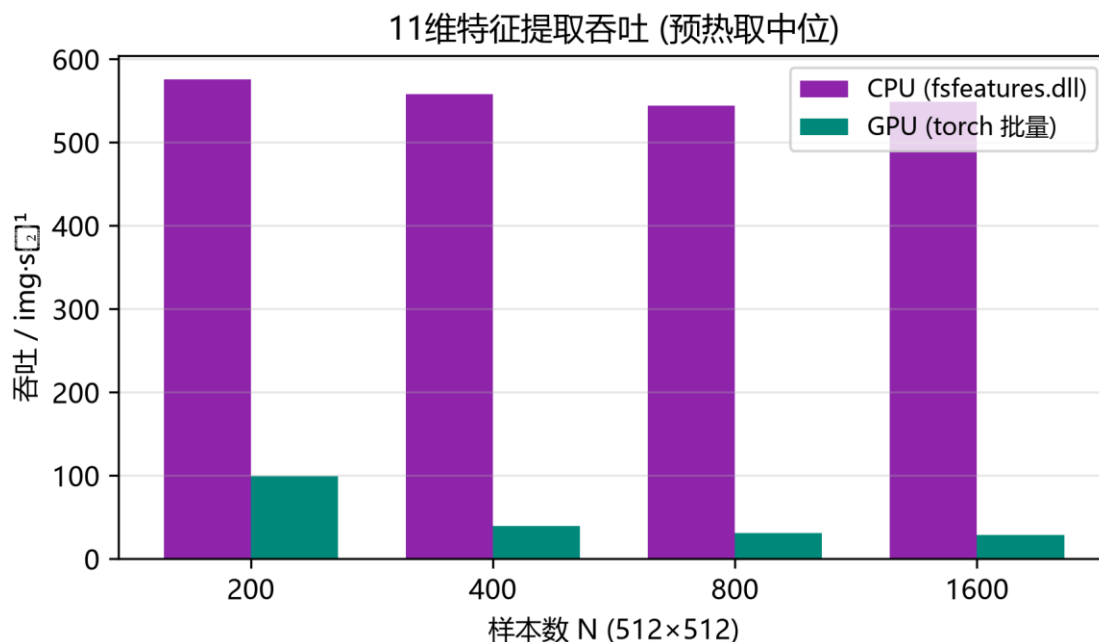


图 9-3 v1 11 维特征提取吞吐：CPU(C++) vs GPU(torch 批量) (项目图)

回到项目看代码 |

读 `gpu/featurize_gpu.py` 的自检函数：它把 GPU 结果与 `src/fsfeatures.py` 的 CPU 结果逐元素比较。**bit 级一致**是这套双实现能并存的前提。再读 README 关于吞吐边界的讨论：小批量时主机↔设

备拷贝开销可能让 GPU 反而不如 C++——这是诚实评测，不是“GPU 一定更快”的营销话术。

9.4 GUI：把研究工具变成可教学的应用

src/gui.py 的主界面围绕“嵌入→解码→分析”三步设计。真正出彩的是两个教学面板：

- **矩阵编码演示** (matrix_demo.py)：随机/手点一个块，实时计算 s、m、d，高亮命中的列与被改系数，直观展示“至多改 1 位”；
- **载荷扫描** (scan_panel.py)：拖动 payload 0→0.4，逐档重新嵌入并刷新卡方 p 值、RS 估计率与 ML 概率三条曲线，观察“藏得越满越容易被发现”。

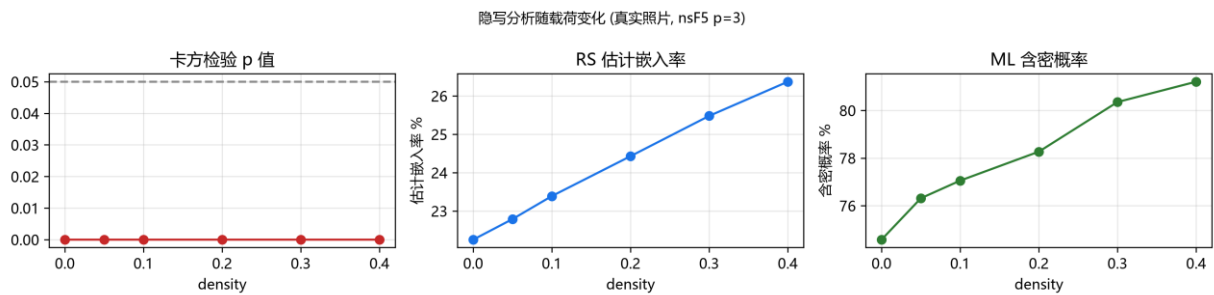


图 9-4 载荷扫描：统计可检测性随嵌入密度变化 (项目图)

避坑提醒 |

注意 GUI 文案里的严谨措辞：单张图没有“AUC”（AUC 需要一组正负样本），所以扫描面板用的是模型概率作为趋势示意。阅读软件输出时，要区分“单图概率”与“数据集指标”，这是容易被误用的一处。

9.5 测试与 CI：让重构不翻车

测试文件	验证内容	心智模型
test_core.py	汉明矩阵正确性、嵌入/解码往返、湿纸、口令错误	算法没坏

测试文件	验证内容	心智模型
test_steg.py	盲分析能区分干净图与含密图	分析没坏
test_false_positive.py	大量干净图不误报（回归）	误报没失控
test_gui.py	窗口能构建、载入与预览	界面没坏

`.github/workflows/ci.yml` 会在每次推送到 main 或提 PR 时自动跑核心测试、构建 wheel 与 sdist；推送 `v*` 标签时自动发 GitHub Release。这套“测试 + 打包 + 自动发布”是算法项目走向可复用工具的标准姿势。

9.6 代码阅读路线图（按需选用）

- 1. 入口：`src/run_e2e.py`（最短路径，串起所有模块）；
- 2. 数据层：`image_io.py` → 数组怎么进出的；
- 3. 嵌入层：`ns5_core.py` 从上到下读一遍（哈希、置换、汉明、湿纸、高层 API）；
- 4. 分析层：`steganalysis.py` 的 `analyze()` 逆向追踪每个统计量；
- 5. ML 层：`train_model.py` 主流程 → `featurize_v2.py` / `srm_filter.py`（v1.4）→ `ml_predict.py` 双版本推理；
- 6. 加速层：`cppembed.py` / `fsfeatures.py` 的 `ctypes` 与自校验；
- 7. 界面层：`gui.py` 找按钮回调，顺藤摸瓜到算法函数。

想一想 |

为什么 `ns5_core.py` 的注释反复强调“改了置换算法会破坏可逆性”？请结合 v1.2.1 修复（DLL 缺失自动回退）想：如果 Python 与 C++ 排列不一致，会出现什么灾难性现象？

9.7 v1.4.0 更新：SRM、多源数据与模型工程（2026-09）

v1.4.0 新增/改动了若干工程模块，阅读新代码时先认目录：

新文件 / 模块	职责	学习要点
----------	----	------

新文件 / 模块	职责	学习要点
src/srm_filter.py	30 个标准 SRM 高通核, numpy/torch 双实现	核清单、归一化、增强图生成
src/featurize_v2.py	143 维 v2 特征组装与 CPU 实现	ALL_FEATURE_NAMES、featurize_v2()
gpu/featurize_v2_gpu.py	GPU 批量 143 维特征与一致性自检	extract_features_v2_gpu()
src/make_dataset.py 扩展	多进程 -j、SRM/feature-set/variants	按图并行、输出逐行一致
src/train_model.py 扩展	DS_FILES 多数据集、Stacking、LGB 调优	双版本模型保存与阈值
gpu/make_imageset.py 扩展	memmap 逐张落盘、--out/--id-offset	多源分工、避免 OOM

数据侧：项目建立了纯校园照片基准 data/campus.jpg (414 张, 移除早期 DIP4E 教材图), 并引入隐写分析事实标准 BOSSbase 1.01 (1 万张 512² 灰度 PGM) 做多源合并。2026-09-14 复核 (可追到 experiments/data/gpu_pipeline_metrics.csv) : GPU 单跑校园 imageset AUC 0.7903 (Youden 阈值 0.713、acc 0.802), BOSSbase 单跑 AUC 0.6438 (0.798 / 0.655); 合并训练因仓库现有的 BOSSbase 图像集为 10000 样本 (早期为 50000), 改按 3410+10000=13410 样本重跑, AUC 0.6672。make_dataset.py 支持多进程 (16 核约 5.6 倍加速), GPU 图像集用 memmap 逐张落盘避免大数组 OOM。

工程配套：许可证从 MIT 改为 Apache-2.0 (新增 NOTICE 与第三方声明), README 与 pyproject 的版本/主页同步为 v1.4.0; C++ DLL 与 Python 的像素级/特征级一致性自检仍然保留。

动手做 | 跑一次 `python gpu\featurize_v2_gpu.py` 自检; 然后按 README 用 data\campus.jpg 与 data\BOSSbase_1.01 各生成一源数据, 对比“单源校园 / BOSSbase 单源 / 两源合并”三组模型的 AUC。这是 v1.4 最值得亲手复现的“数据域”实验。

第 10 章 · 综合实践：从“读懂”到“做出来”（第 11–12 周）

动手做 |

本章目标：用两周时间完成一个可演示的改进实验。下面给出三个由易到难 的方向与验收清单，你也可以把它们组合起来。

10.1 方向 A：复现并批判性验证（最稳）

选 README / docs/RESULTS.md 中的 1 ~ 2 个结论，自己重新跑并尝试“推翻”它：

- 复现码族图：理论 $\alpha(p)$ 是否与实测吻合？ p 取大后偏差从哪来？
- 复现检测实验：换你自己的照片训练，比较 v1 11 维基线（约 0.75–0.79）与 README 的 v1.4 双版本结果，观察照片风格对 AUC 的影响；
- 复现篡改实验：改 1 像素、改 1 行、改 1 块，解码失败率各是多少？

验收项	通过标准
实验日志	记录环境、命令、参数、原始输出
图表	至少 2 张自绘曲线/对比图，坐标与图例完整
结论	能区分“验证了论文”与“与论文不一致”并给原因
可复现	提供一键脚本或逐条命令，别人能重跑

10.2 方向 B：功能扩展（最有成就感）

- **支持中文/UTF-8 消息**：v1.8 起已在 `encode_string()` 内实现（长度头单位是字节数），可对照源码校对你的实现；
- **彩色通道扩展**：目前仅用 R 通道，可研究 R/G/B 三通道如何分配容量与安全性；
- **新增特征**：在 v1 11 维或 v2 143 维基础上加一个你认为能区分 clean/stego 的统计量，用 GroupKFold 评估 AUC 是否真的上升；

- **对比新算法：**实现朴素 LSB 替换，与 nsF5 在相同载荷下比较 Gn 塌缩与检出率。

避坑提醒 |

每次扩展都要跑 `test_core.py` 与 `test_steg.py`，保证“嵌入→解码”往返仍然成立。修改编码方案时，解码端必须同步修改——这是初学者最常忘的。

10.3 方向 C：教学演示再包装

把 GUI 里的矩阵编码演示扩展成一套可讲解的教材页/动画脚本，例如：

- 逐步动画：随机块 → 显示 H → 算 s → 给 m → 求 d → 高亮 → 校验；
- 题库模式：随机出题让你手算“该翻转哪一位”，自动判分；
- 对比模式：同一消息分别用 LSB、matrix p3、nsF5 p3 嵌入，并排显示改动数与 Gn。

10.4 文档与代码对照阅读表

论文章节	对应代码	读它之前先掌握
第 2 章 技术基础	ns5_core.py / steganalysis.py	LSB、汉明码、湿纸、卡方/RS
第 3 章 需求分析	README 功能总览	隐写与隐写分析博弈观
第 4 章 总体设计	arch 图 + ns5_core.py	分层架构、哈希键控流程
第 4.5 节 加速	cppembed.py / fsfeatures.py	ctypes 与一致性自校验
第 4.6 节 GPU	gpu/*.py	张量化、批量、吞吐边界
第 5 章 实验	run_e2e / make_dataset / train_model	GroupKFold、AUC、Youden

提示：本项目不再随仓库分发论文稿；对外数字一律以 `docs/RESULTS.md`（权威结果表）为准，它逐行标注了语料、协议与是否可溯源。下表按早期论文章节举例，阅读时请对照该表。

10.5 汇报/答辩常见问题与应答要点

高频问题	应答要点
LSB 隐写为什么能被发现？	相邻灰度对被拉平（卡方）+ LSB 结构塌缩（RS），先讲直觉再讲统计量
矩阵编码为什么“改得少”？	$n=2^p-1$ 个位置承载 p 位， H 列互异 $\rightarrow$ 任一伴随式差对应唯一位置
nsF5 相比 F5 好在哪里？	湿纸预标记湿点、只在干点解方程，无收缩无重嵌
湿纸编码怎么解方程？	GF(2) 线性方程 $H_{dry} \cdot y = d$ ，先试单列/双列，再高斯消元兜底
为什么不用裸 CNN？	小样本弱信号下学不到泛化特征；特征+简单模型更稳（AUC 实证）
AUC 0.756 说明什么？	排序能力尚可但弱密度难检；AUC 与“具体阈值下检出率”是两回事
哈希键控能防什么？	防位置可预测/复制；能感知普通篡改，但不是密钥化 MAC
C++/GPU 加速有没有风险？	有：必须做跨语言一致性校验，否则嵌入解码会不可逆

10.6 交付清单（最后一周）

- 改进代码 diff（自己能讲清每处改动）；
- 实验脚本 + 原始输出（不加工、不挑数）；
- 2~3 张图表（效率/ROC/密度检出/篡改任选）；
- 3~5 页报告：问题→方法→结果→局限→下一步；
- 5 分钟演示：从嵌入到分析的一条龙运行；
- 能脱稿回答 10.5 表中的任意三个问题。

第 11 章 · 边界、伦理与继续前进（穿插学习）

11.1 法律与伦理边界

隐写是双刃剑：它支撑版权水印、媒体溯源、隐蔽认证等正当应用，也可能被用于 恶意隐蔽通信。

学习与研究的正确姿势是：

- 只在自有数据、公开数据集或经授权场景做实验；
- 把工具用于“检测与取证”而非“协助隐藏”；
- 发表结果时说明局限与误报风险，不夸大检测能力；
- 不把他人的隐私图像当作练习载体（照片数据要脱敏、授权）。

避坑提醒 |

这个项目的隐写分析部分（卡方/RS/ML）正是“盾”的用途：监管与取证需要它来发现可能的隐蔽信道。研究攻击面之前，先想清楚 你的实验数据与使用目的。

11.2 项目已知局限（看懂边界才算真正读懂）

- 载荷支持 UTF-8 文本（中文可直接嵌入）；更实际的限制是仅支持文本、不支持文件；
- 彩色图只使用 R 通道，容量利用率低；
- 内容是哈希键控而非密钥化 MAC，无法防“重嵌并更新头部”的强攻击者；
- v1 数据集的独立源图仍有限（414 张校园 + BOSSbase 后扩到 1 万张）；弱密度与跨源仍是检出难点，任何新模型都要先做 OOD 测试；
- 启发式分析输出的是“隐写倾向概率”，不是严格统计意义上的真实概率；
- 像素域 nsF5 是有损格式的天敌——JPEG 域需要 yccstego 那样的 DCT 系数实现（姊妹项目，独立仓库与 PyPI 包：<https://github.com/Yukinoshita-lin/yccstego>）。

- v2/143 维模型对训练分布外的真实 JPEG 干净图曾“过激”，修复依赖把 JPEG 干净样本加入训练；任何模型都要先做 OOD 测试再部署——本项目现口径是 1514 张真实干净照片上 143d 误报 9.58%、53d 28.86%；
- SRM 只在同源数据上提升检测，跨异构源反而下降，预处理收益有严格适用范围；
- 双版本模型各有取舍：143d 更准（8-split 平均 AUC 0.8980、弱档检出 50.0%、真实干净照片 OOD 误报 9.58%）；53d 去掉 SRM 后每一维都能解释，代价是 AUC 约低 0.05（0.8461 / 41.4% / 28.86%）。GUI 切换仍在规划，当前用 API 指定 model_path；
- 项目许可证已改为 Apache-2.0 并含 NOTICE，分发/引用时需保留第三方声明。

11.3 现代信息隐藏与隐写分析地图

方向	代表思想/方法	与本科生的距离
内容自适应嵌入	HUGO / WOW / UNIWARD：在纹理复杂处嵌入	需要较深最优化背景
深度学习隐写	编码器-解码器端到端训练隐藏模型	可做入门复现
深度学习隐写分析	SRNet / 大规模 CNN 检测	需要大算力与大数据
JPEG 域隐写	yccstego: Y 通道量化 DCT 系数 nsF5（姊妹项目，独立仓库： https://github.com/Yukinoshita-lin/yccstego ）（姊妹项目，独立仓库： https://github.com/Yukinoshita-lin/yccstego ）	本项目即可扩展
可逆/鲁棒水印	兼顾不可见性、鲁棒性与容量	工程性强，适合就业方向

如果你的兴趣被点燃，推荐的下一步：把本手册第 5 章“湿纸”读进原始论文（Fridrich et al.），再用姊妹项目 yccstego（<https://github.com/Yukinoshita-lin/yccstego>）对照学习 JPEG 量化域，最后挑一个现代算法做文献综述——附录 E 给了具体入口。

11.4 结语

刚开始时你也许以为隐写就是“把字藏进图片”，现在你应该能讲出：它为什么依赖 载体冗余、为什么直接改 LSB 会留下统计指纹、汉明码如何用最少的改动传递信息、湿纸编码如何让嵌入“无收缩”，以及机器学习如何诚实地说出“我有多确定”。这些知识串起来的，不只是 F:\Steganography 这一个项目，而是信息安全、概率统计与机器学习交汇处的一种思维方式。

动手做 |

最后做一个收尾练习：不看任何资料，用 500 字向一位同学介绍“nsF5 为什么比朴素 LSB 更好”。写完后打开 README 对照，查漏补缺——能写清，才算真正学完。

附录 A · 术语中英对照

按出现顺序整理，方便随时查阅。掌握粗体项即可覆盖本手册绝大部分内容。

中文	英文	一句话解释
隐写术	Steganography	把秘密信息藏进正常载体，隐藏“通信行为本身”
隐写分析	Steganalysis	通过统计/学习判断载体是否藏有秘密
载体 / 含密图	cover / stego	嵌入前的原始图像 / 嵌入后的图像
最低有效位	LSB	像素二进制的最低位，翻转后视觉几乎不变
位平面	bit plane	按二进制位拆出的二值图层，共 8 层
冗余	redundancy	载体中可被修改而不易察觉的部分
嵌入容量	capacity / payload	一张图最多能承载的秘密比特数
相对载荷	relative payload	嵌入比特数与可嵌位置数之比
嵌入效率	embedding efficiency	平均每次修改所携带的比特数 α
矩阵嵌入	matrix embedding	用分组编码在更少改动中嵌入更多位
二元汉明码	binary Hamming code	能纠正/定位 1 位错误的线性码 $[n,k,3]$
校验矩阵	parity-check matrix H	$p \times n$ 矩阵，列互异，用于算伴随式
伴随式	syndrome	$s = H \cdot x \pmod{2}$ ，块的“现状摘要”
GF(2)	Galois field of 2	只含 0/1、按异或运算的有限域
收缩	shrinkage	减幅时系数变 0 导致块作废的现象
湿纸编码	wet paper coding	湿点不动、只在干点解方程完成嵌入
干点 / 湿点	dry / wet positions	可修改的位置 / 碰不得的位置
F5 / nsF5	F5 / nsF5 algorithm	JPEG 矩阵嵌入；nsF5 用湿纸消除收缩
减幅	coefficient shrinking	让系数绝对值减小 1 的修改方式
SHA-256	SHA-256	输出 256 位摘要的密码学哈希函数

中文	英文	一句话解释
键控	keying	由口令/内容密钥决定隐藏位置等行为
确定性置换	deterministic permutation	同种子必得同排列的洗牌算法
splitmix64	splitmix64	项目使用的 64 位伪随机数生成器
Fisher-Yates	Fisher-Yates shuffle	等概率洗牌的标准算法
自同步	self-synchronization	解码端无需外部参数即能找到正文
篡改感知	tamper perception	图像被改动后能够被察觉
盲隐写分析	blind steganalysis	无原始载体时的统计检测
卡方检验	chi-square test	检验相邻灰度对是否被拉平
p 值	p-value	观测与假设吻合的概率（此处高 p 反而可疑）
RS 分析	RS analysis	通过正负掩码扰动统计规则/奇异组
常规/奇异组	regular / singular	扰动后判别值变大 / 变小的像素组
差分熵	difference entropy	相邻像素差分布的香农熵，衡量纹理随机度
香农熵	Shannon entropy	信息量/不确定度的度量，单位 bit
监督学习	supervised learning	用带标签样本训练模型
特征向量	feature vector	描述样本的数值列表（本项目 11 维）
标签	label	样本的正确答案（0=干净，1=含密）
二分类	binary classification	输出属于两类之一的预测
逻辑回归	logistic regression	线性加权 + sigmoid 的可解释分类器
sigmoid	sigmoid	把任意实数压到 0~1 的 S 形函数
交叉熵损失	cross-entropy loss	分类常用的损失函数
梯度下降	gradient descent	沿损失下降方向迭代更新参数
过拟合	overfitting	背下训练样本而失去泛化能力

中文	英文	一句话解释
交叉验证	cross-validation	轮流用不同折验证模型的流程
数据泄漏	data leakage	验证信息混入训练导致指标虚高
GroupKFold	GroupKFold	按组切分的交叉验证（同照片样本同组）
混淆矩阵	confusion matrix	真实 vs 预测的四格计数表
精确率/召回率	precision / recall	判对比例 / 抓全比例
ROC / AUC	ROC curve / AUC	全阈值下的检出-误报曲线及其面积
Youden 准则	Youden' s J	在 tpr-fpr 最大处选阈值
误报 / 漏报	false positive / negative	把干净判成含密 / 把含密判成干净
ctypes	ctypes	Python 调用 C/C++ 动态库的接口
DLL	dynamic-link library	Windows 动态链接库
CUDA / 批处理	CUDA / batching	GPU 并行计算 / 多样本同时处理
一致性校验	consistency check	C++/GPU 与 Python 结果逐位比对
ASCII / UTF-8	ASCII / UTF-8	英文字符编码 / 通用字符编码（含中文）
空间富模型	SRM / Spatial Rich Model	一组高通滤波残差核，用于突出嵌入噪声
残差图	residual map	高通滤波后保留的“噪声残差”
LightGBM	LightGBM / LGB	梯度提升树实现（v1.4 双版本模型使用的分类器）
特征组	feature group	同一来源的一批特征（BASE/SRM/PREFIX/LSB-PREFIX 等）
分布外	out-of-distribution (OOD)	与训练分布不同的输入（如真实 JPEG 干净图）
BOSSbase	BOSSbase 1.01	隐写分析标准基准库：1 万张 512 ² 灰度 PGM
PGM	PGM	无损灰度图像格式（BOSSbase 载体格式）
Stacking	stacking / meta-learner	用次级模型组合多个基模型预测的集成方法
内存映射	memmap	大数组分批落盘、按需读写，避免一次性 OOM

中文	英文	一句话解释
校准集	calibration set	单独用于选择阈值的样本子集
网格调优	grid tuning	在超参数网格上搜索并比较模型
双版本模型	dual-version model	143d 稳健版 + 53d 可解释版并存部署策略

附录 B · 常用命令速查

以下命令均在项目根目录 F:\Steganography 下执行。

目的	命令
安装核心依赖	<code>pip install numpy pillow</code>
安装全部依赖 (ML/绘图)	<code>pip install -e . 或 pip install numpy pillow matplotlib scikit-learn joblib pandas</code>
核心算法自测	<code>python src\test_core.py</code>
隐写分析自测	<code>python src\test_steg.py</code>
误报回归测试	<code>python src\test_false_positive.py</code>
GUI 冒烟测试	<code>python src\test_gui.py</code>
端到端验证	<code>python src\run_e2e.py</code>
启动图形界面	<code>python src\gui.py</code>
生成码族/效率图	<code>python -c "import sys; sys.path.insert(0,'src'); from efficiency import plot_code_family_and_efficiency; print(plot_code_family_and_efficiency())"</code>
生成特征数据集	<code>python src\make_dataset.py data\campus.jpg --out campus</code>
训练/评估分类器	<code>\$env:DS_FILES = "dataset.csv"; python src\train_model.py</code>
单图 ML 判定	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import get_predictor; import numpy as np; from PIL import Image; print(get_predictor().predict(np.asarray(Image.open('img/cover.png')).convert('L')))"</code>
C++/Python 一致性自检	<code>python -c "import sys; sys.path.insert(0,'src'); import cppembed; cppembed.selfcheck()"</code>
GPU 数据集生成	<code>python gpu\make_imageset.py</code>
GPU 特征自检	<code>python gpu\featurize_gpu.py</code>

目的	命令
GPU 训练	<code>python gpu\train_ml_gpu.py</code>
GPU 单图检测	<code>python gpu\predict_gpu.py img\cover.png</code>
构建发布包	<code>python -m build</code>
安装 wheel	<code>pip install dist\nsf5stego-1.8.4-py3-none-any.whl (或直接 pip install nsf5stego)</code>
生成 v2 数据集 (12 档 / 143 维)	<code>python src\make_dataset.py --out campus_v2 --feature-set v2 --variants all</code>
生成 SRM 增强数据集 (同源)	<code>python src\make_dataset.py data\campus_jpg --out campus_srm --preprocess srm -j 16</code>
指定 v2+JPEG 数据集训练	<code>\$env:DS_FILES = "dataset_campus_v2_jpeg.csv"; python src\train_model.py</code>
GPU v2 特征自检	<code>python gpu\featurize_v2_gpu.py</code>
BOSSbase 多源生成 + GPU 训练	<code>py gpu\make_imageset.py data\BOSSbase_1.01 --out bossbase (再运行 python gpu\train_ml_gpu.py)</code>
切换 53d 可解释模型 (API)	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import MLPredictor; import numpy as np; from PIL import Image; print(MLPredictor(model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib', clip_outliers=False).predict(np.asarray(Image.open('img/cover.png').convert('L'))))"</code>

附录 C · 12 周打卡检查表

每完成一项就在日期列写下当天日期。两周内完成一章不丢人，跳过动手实验才丢。

周	主题	必须完成的动作	日期
1	图像与二进制	运行位运算示例；说出 LSB 定义	
2	Python 环境	pip 装依赖；跑通 run_e2e.py；保存一张灰度图	
3	LSB 与盲分析	嵌入一分析实验；看懂 test_steg.py 的输出	
4	卡方/RS 原理	给同学讲清 Gn 塌缩；跑一遍 test_false_positive.py	
5	矩阵编码/F5	手算一个 $p=3$ 例子；用 matrix_demo 验证	
6	nsF5/湿纸	读 ns5_core.py 湿纸段；跑 test_wet_paper_needed	
7	哈希键控	口令错误实验；篡改 1 像素实验；写出键控边界	
8	ML 基础	跑通 v1 训练；能解释特征/泄漏/GroupKFold/AUC	
9	ML 隐写检测	对比 11d/143d/53d 判定；解释 SRM 与双版本策略	
10	工程化	跑 cppembed.selfcheck 与 featurize_v2_gpu 自检；读 SRM/多源管线	
11	综合项目	选定方向并完成 70% 代码/实验	
12	汇报	完成报告与演示；脱稿回答 10.5 的三个问题	

附录 D · 概念 → 代码定位表

学习后期“忘了某概念在哪个文件”，查这张表最快。

概念	文件	优先阅读内容
消息编码/长度头	src/ns5_core.py	encode_string / decode_string
图像哈希/种子	src/ns5_core.py	derive_seed / get_image_hash
确定性置换	src/ns5_core.py	permute_index
汉明码/伴随式	src/ns5_core.py	build_hamming / syndrome
矩阵嵌入	src/ns5_core.py	MatrixEmbedding._embed
湿纸求解	src/ns5_core.py	solve_wet_paper / gauss_solve_GF2
像素域 nsF5	src/ns5_core.py	nsF5Pixel._embed
高层嵌入/解码	src/ns5_core.py	embed_string / extract_string
卡方统计	src/steganalysis.py	chi2_stats / chi2_sf
RS 统计	src/steganalysis.py	rs_metrics
综合判读	src/steganalysis.py	analyze
特征顺序	src/fsfeatures.py / ml_predict.py	FEAT_KEYS
训练流水线	src/train_model.py	main()
数据集生成	src/make_dataset.py	main()
C++ 嵌入封装	src/cppembed.py	embed_string / selfcheck
GPU 特征	gpu/featurize_gpu.py	批量算子与 check_vs_cpu
GUI 回调	src/gui.py	按钮事件 → 核心函数
v2 特征计算	src/featurize_v2.py	ALL_FEATURE_NAMES / featurize_v2()
SRM 预处理	src/srm_filter.py	srm_residuals_np / preprocess_batch_torch

概念	文件	优先阅读内容
模型推理（双版本）	src/ml_predict.py	MLPredictor(model_path=..., clip_outliers=...)
GPU v2 特征	gpu/featurize_v2_gpu.py	extract_features_v2_gpu() 与自检

附录 E · 推荐资源

书籍（中文先入门）

- 《机器学习》（周志华，清华大学出版社）——第 2 章“模型评估与选择”务必精读；
- 《统计学习方法》（李航）——逻辑回归与感知机的经典推导；
- 《数字图像处理》（Gonzalez & Woods）——图像处理经典教材：位平面、直方图与频域基础（早期实验用过其中教材图，v1.4 数据源已改为纯校园照片与 BOSSbase）；
- 《深入浅出信息隐藏》类中文教材可作为扩展阅读。

核心论文（按阅读顺序）

- Westfeld, “F5—A Steganographic Algorithm”, 2001——F5 与矩阵嵌入出处；
- Westfeld & Pfitzmann, “Attacks on Steganographic Systems”, 1999——卡方检验；
- Fridrich, Goljan & Du, “Reliable Detection of LSB Steganography in Color and Grayscale Images”, 2001——RS 分析；
- Fridrich, Goljan, Lisonek & Soukal, “Writing on Wet Paper”, 2005——湿纸编码；
- Pevný, Filler & Bas, “Using High-Dimensional Image Statistics to Perform Unsigned Steganalysis”——现代特征法（SPAM）入口。

在线资料与工具

- 项目 README 与权威结果表 docs/RESULTS.md（SRM/143d/53d 与多源实验的现口径，附语料与协议标注）；GitHub: Yukinoshita-lin/nsf5-steganography；
- scikit-learn 官方文档：LogisticRegression、GroupKFold、roc_curve；
- PyTorch 官方教程（GPU 版依赖）；
- GitHub Actions 文档：了解 CI 中如何跑测试与发版。

动手做 |

到这里，手册主体结束。回到导读 0.4 的能力清单，逐条给自己打分：全部能打勾，就给自己发一张“nsF5 入门结业证”吧。

附录 H · 命令行工具与安装版 (v1.8.4)

本手册正文以“从 Python 代码出发”讲解算法; v1.8 起, 项目同时提供不写一行代码就能完成完整流程的两种形态: 图形界面 (见第 2 章与第 10 章) 与命令行工具 nsf5stego。本章速查这两种形态的安装与用法, 命令在 Windows PowerShell 与 Linux/macOS 终端下均可运行。

H.1 三种安装方式

PyPI (推荐 pip 用户): `pip install nsf5stego` —— 自带两个部署模型、命令行入口与图形界面;
Windows 安装版: 到 GitHub Releases 下载 `nsf5stego-setup-1.8.4.exe`, 简体中文向导、每用户免管理员、开始菜单快捷方式、可选加入 PATH; 免安装便携包: 解压 `nsf5stego-portable-1.8.4-win64.zip` 即用。三者算法完全一致, 按使用场景任选。

H.2 命令行三步 (与 GUI 一一对应)

嵌入 (把文本写进图片):

```
nsf5stego embed cover.png -m "要隐藏的文本" -p 口令 -o stego.png
```

解码 (从含密图还原文本, 方法/p/口令须与嵌入一致):

```
nsf5stego extract stego.png -p 口令
```

盲隐写分析 (卡方 + RS + 机器学习判定; 支持多图批量):

```
nsf5stego analyze stego.png --json
```

文本按 UTF-8 编码, 中文可直接嵌入; analyze 支持 --json 机器可读输出与通配符批量, 退出码 0/1/2 区分 成功/失败/参数错误, 便于脚本化考核。

H.3 新手引导路径

首次打开图形界面会自动载入一张演示封面, 界面顶部的引导条会随操作推进, 始终提示 “下一步点哪里” : 点「嵌入并保存」(观察改动像素数与保存路径) → 勾选「查看差异」放大看哪些像素变了 → 点「解码提取」验证完整还原。整个闭环一分钟内可完成, 完成后再回到正文对应章节理解背后的原理。

H.4 与手册正文的对应

命令行的实现入口是 src/cli.py, 算法核心仍在 ns5_core.py: 嵌入/解码原理见第 1–4 章, 盲隐写分析见第 5 章, 机器学习检测见第 8–9 章。用 CLI 完成实验后回到对应章节阅读源码, 学习路径与纯 GUI 用户完全一致。